Compiler repeated work: profile, reduce, verify

Created-at: 2026-09-07 12:48 +02:00

Scope and ETA. Investigate cold source compilation, AST copying, reference resolution and repeated traversals. Approximately 45 minutes: 10 minutes instrumentation, 10 minutes profiles and baseline, 15 minutes implementation and correctness, 10 minutes repeat measurements and report. Generation, parsing, lowering, hashing, prepared compilation and destruction remain distinct measurement scopes. Do not remove validation or change hashes.

Authority and checkout. User requested optimization and investigation of repeated calls. Core fetched successfully at 12:42–12:45 using the existing origin with an SSH HostName override for 192.168.1.228; normal DNS failed. Base is freshly fetched origin/main f305ade; branch perf/compiler-repeated-work reuses the existing checkout. Former branch 2d43e82 has three unmerged diagnostic commits and is preserved. Untracked distributed-verification artifacts are preserved.

Discovery. Extend the existing interning Criterion benchmark and ModuleStore integration tests. ir_bench explicitly excludes parsing/lowering; ir_depth_probe measures depth failure, not compilation. Historical CPU optimization and CBOR reports contain no equivalent compiler profile. Ecosystem reader confirms Core owns compilation and is consumed by Graph, Transformation and Python.

Affected files. This four-format plan; `hymeko_core/benches/interning_bench.rs`; `hymeko_core/tests/mir` candidate production files `hymeko_core/src/module_store/module_store.rs`, `hymeko_core/src/ir/lower.rs` `hymeko_core/src/ir/canonical_hash.rs`, `hymeko_core/src/resolution/resolve.rs`; `reports/2026-09-07` and associated numerical, profile and figure evidence under `reports/compiler-repeated-work/`.

Core protection. User approved at the 2026-09-07 continuation: `APPROVED-CORE-EDIT: compiler-repeated` Instrumentation is allowlisted; candidate production files are lockdown-full. Apply the prepared root-AST/dependency-list borrow patch first, keeping imports, hashing and validation unchanged. Compare exact IR output (only timestamp normalized), then cold compilation and cached lookup. Preserve before/after binaries and PDBs. The user's runtime question also requires quoting the existing prepared Dijkstra results separately: compiler speedups are not traversal-kernel speedups.

Interfaces and invariants. Retain public signatures, cache behavior, resolution ambiguity/error order, import namespace order, IR contents and hash bytes. The legacy Merkle pass writes serialized decl_hash and is not redundant merely because another hash pass exists. No dependency changes. If borrowing ASTs requires an API change, keep the public owned-slice API and share the underlying lowering.

Measurement and tests. Criterion release, minimum five post-warmup iterations; save raw samples, median/IQR/worst. Windows samply CPU profile with symbols before selecting a hot spot. Read existing Graph campaign artifacts first. One large production-scale smoke precedes

repeated large runs. Record process peak working set, host and toolchain. Differential whole-IR checks normalize only metadata timestamps and pin both canonical and declaration hashes; import, constants, aliases, empty inputs and failure cases. Run relevant and workspace tests, fmt, clippy, symbols, duplication and preflight. Benchmark same machine and build settings.

Budgets and risk. Maximum 16 GiB RSS, 180 seconds per measured process; the existing 50,000-node source provides a quick discriminating workload, followed by the graph map512 source (30,032,345 bytes) if available through the existing harness. Target measurable reductions without greater than 10 percent time or memory regression. No claimed speedup without profile attribution. Nested imports and ambiguous reference paths can invalidate naive early return or memoization; tests must preserve them. Cloning may contribute memory rather than dominant CPU cost. Retaining cache entries may exchange speed for unbounded memory, so do not extend cache lifetime speculatively.

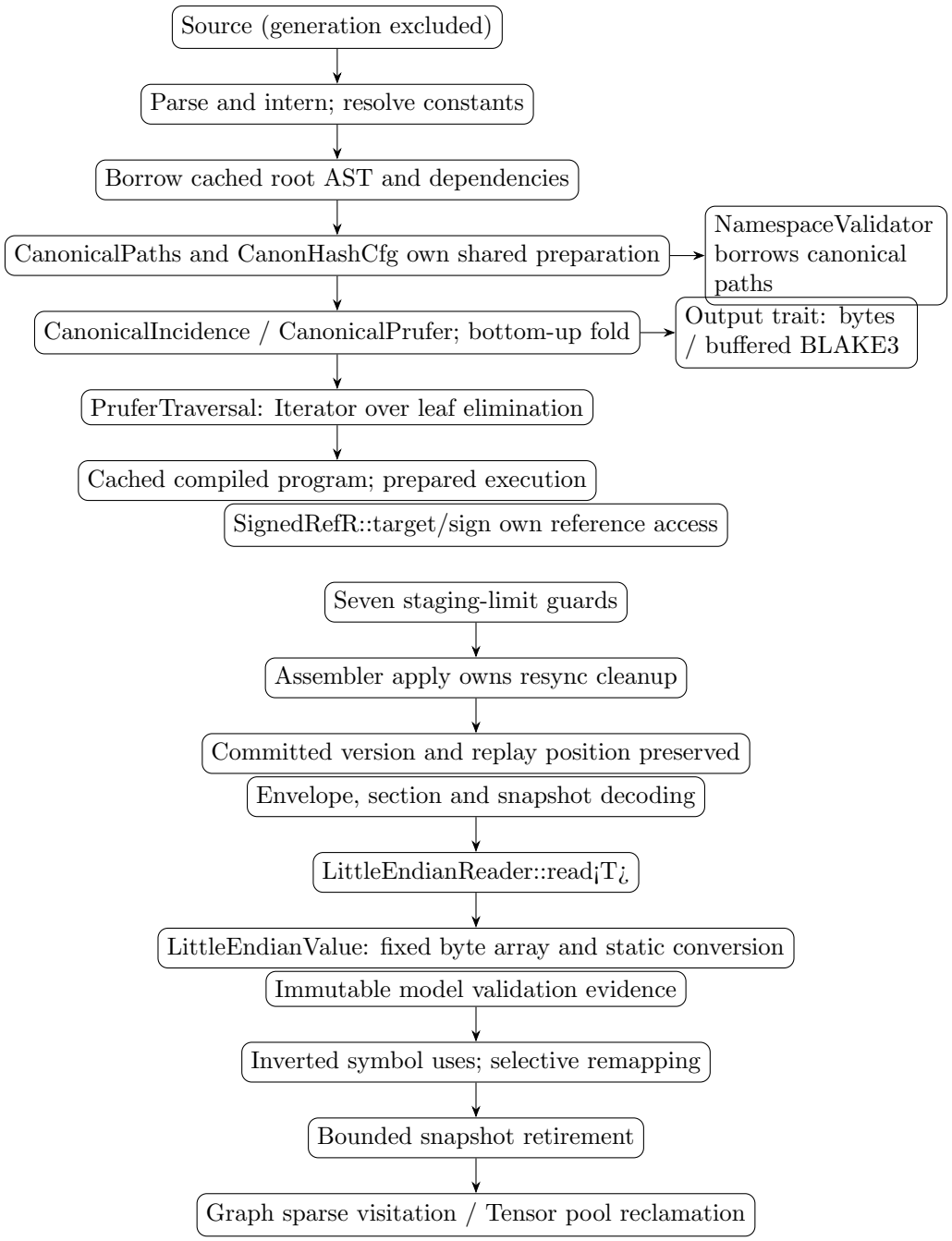
Rollback. Revert only this task's named changes. Preserve user files, previous branches, benchmark data and public hash/wire contracts.

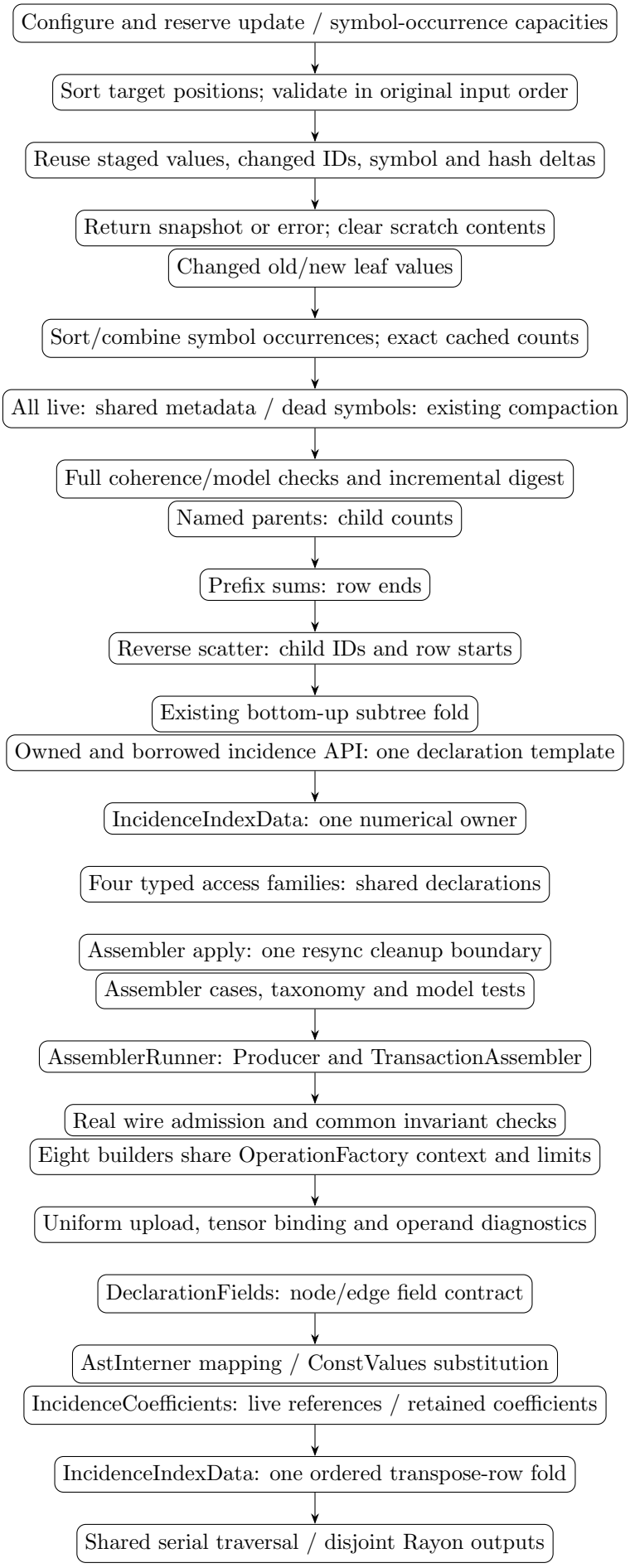
Continuation, 2026-09-07 13:36. Under the same approved slug, strengthen source-to-IR compilation by reusing the canonical path table between validation and hashing. A crate-private CanonicalPaths view borrows the IR and owns just its derived paths; the IR cannot change during its lifetime. Public standalone APIs delegate to the same methods. Validation borrows index paths rather than cloning them, preserving the original index iteration and first-error order. Existing DerivedPathIndex also sorts a lookup table and is therefore inappropriate for this transient compile phase. Extend existing ModuleStore tests to compare compiled and standalone hashes, and to exercise namespace corruption. Frozen hashes and full serialized-IR goldens remain unchanged. Compare the already optimized root-borrow binary against this one change first; only then consider repeated leaf hashing. Expected additional work: 20 minutes including two bounded measurement pairs, tests, coverage and report. Retain the 50k baseline and add a larger existing frontend-generator scale if the smoke fits the declared 16 GiB / 180-second process budget. No second benchmark framework.

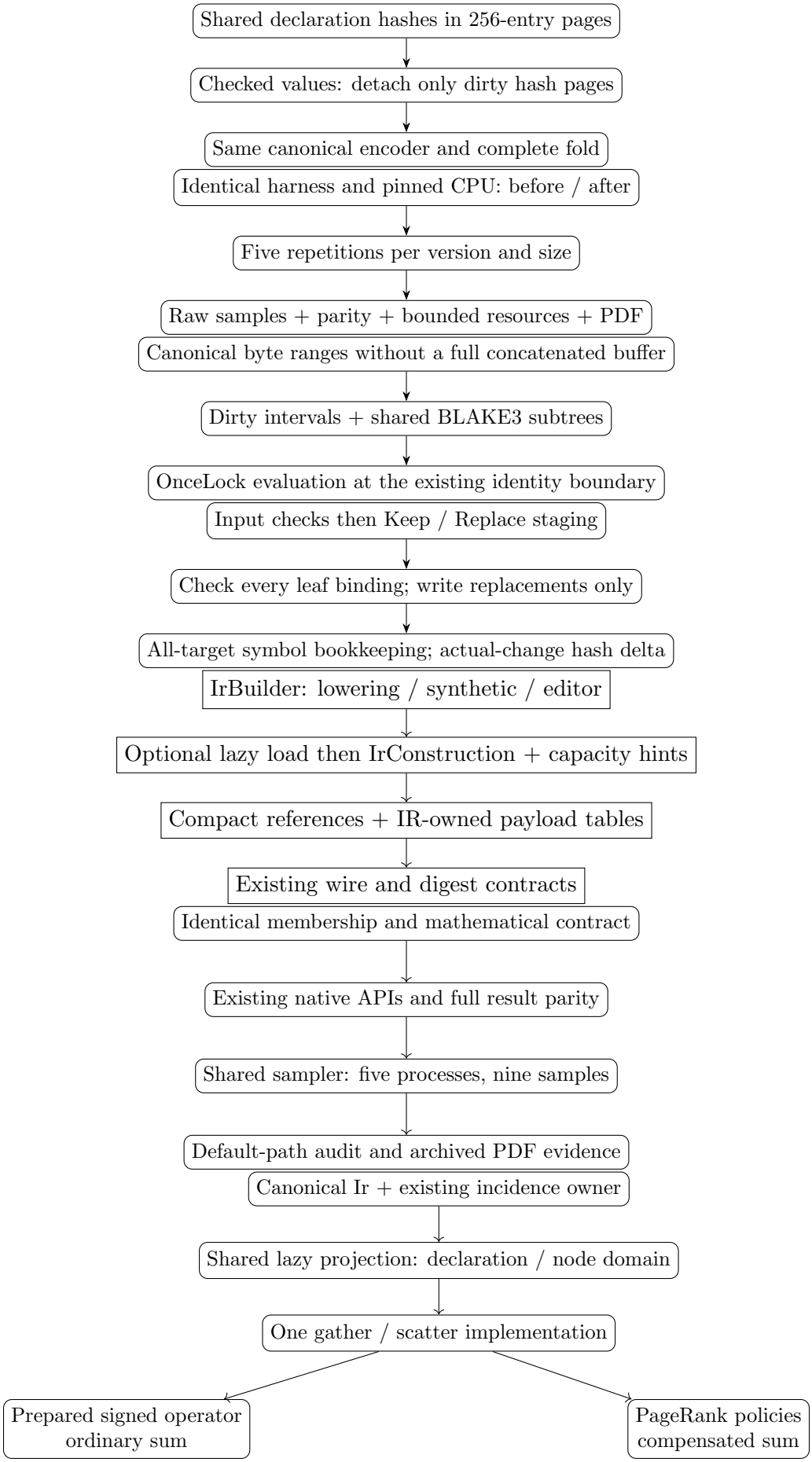
Continuation, 2026-09-07 13:59. Profile-guided compact Pruefer state: the resolved final profile puts 19.1 percent of compile-stack samples in prufer_from_forest. Replace its HashMap of dense declaration IDs with a dense vector and per-vertex adjacency lists with degree plus neighbor XOR. For degree one, the XOR equals the sole remaining neighbor, and removing an edge XORs out its endpoint. Preserve the existing heap key/order, labels, token sequence, super-root handling and hash domains. No public API or dependency change; same Core approval. Extend hymeko_core/tests/unified_ir/verification/frozen_hash.rs before production edits with deterministic chains, stars, balanced trees, multiple roots, empty and single-node forests, interleaved anonymous declarations and nondefault hash configuration. Freeze an aggregate digest against the current shared-path version before the rewrite. Reuse its archived binary as the immediate baseline. Measure cold compile and standalone canonical hashing at 50k, then cold compile at the already validated 500k scale in ABBA order; caps remain 180 seconds and 16 GiB per process. Capture a resolved after-profile, execute workspace tests, clippy and coverage, and append to the same report. ETA about 15 minutes: targeted parity/build 3, bounded A/B 6, profile plus checks/report 6. Roll back only this continuation if parity fails or investigation finds a regression. Existing root-borrow and shared-path edits remain.

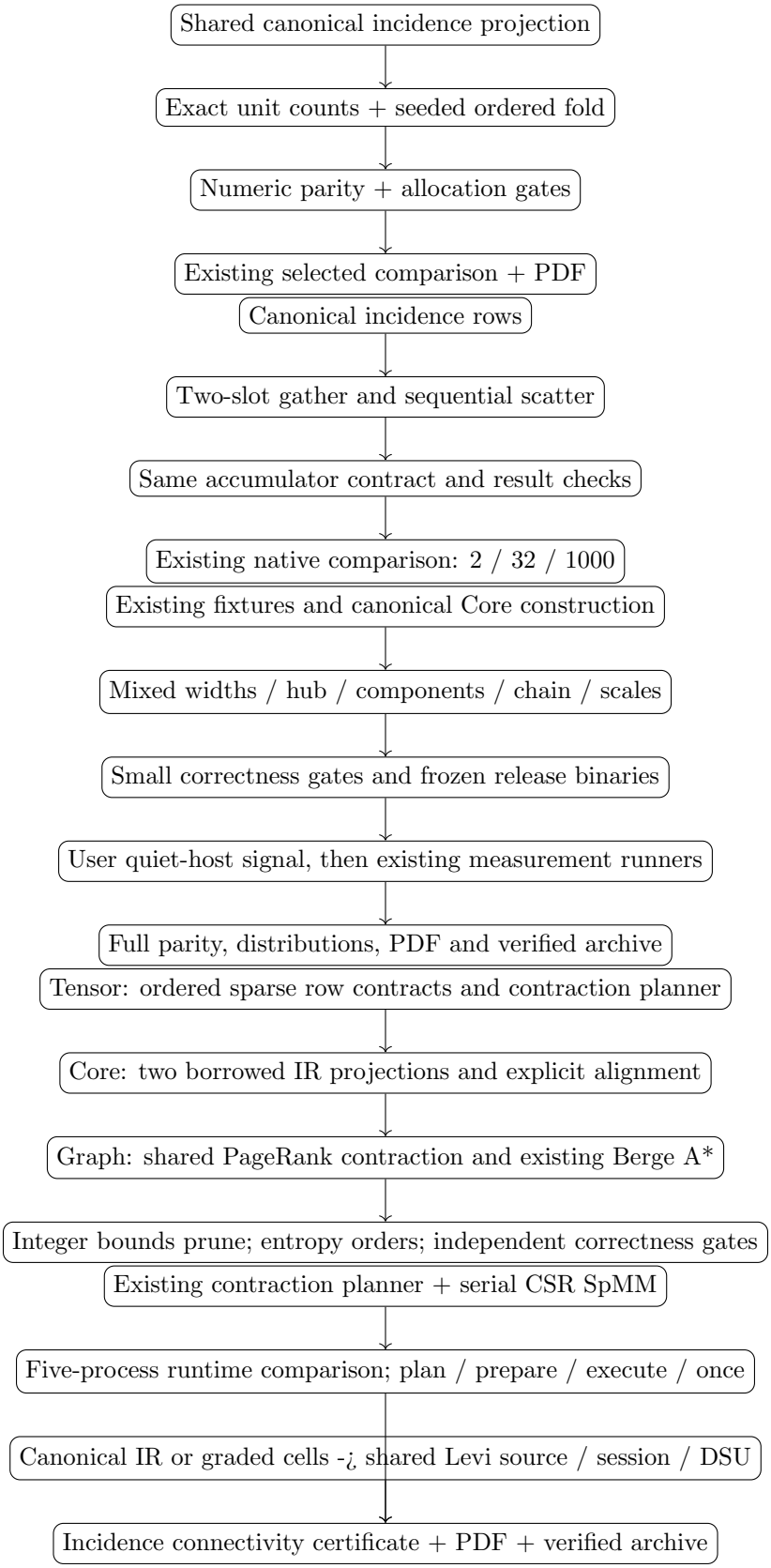
Continuation, 2026-09-07 20:06. Remove duplicate annotation-value resolution in resolve_signed_refs under the existing `APPROVED-CORE-EDIT: compiler-repeated-work`. Fresh fetch confirms HEAD equals origin/main f305ade; preserve all earlier dirty changes. The exact current profile attributes

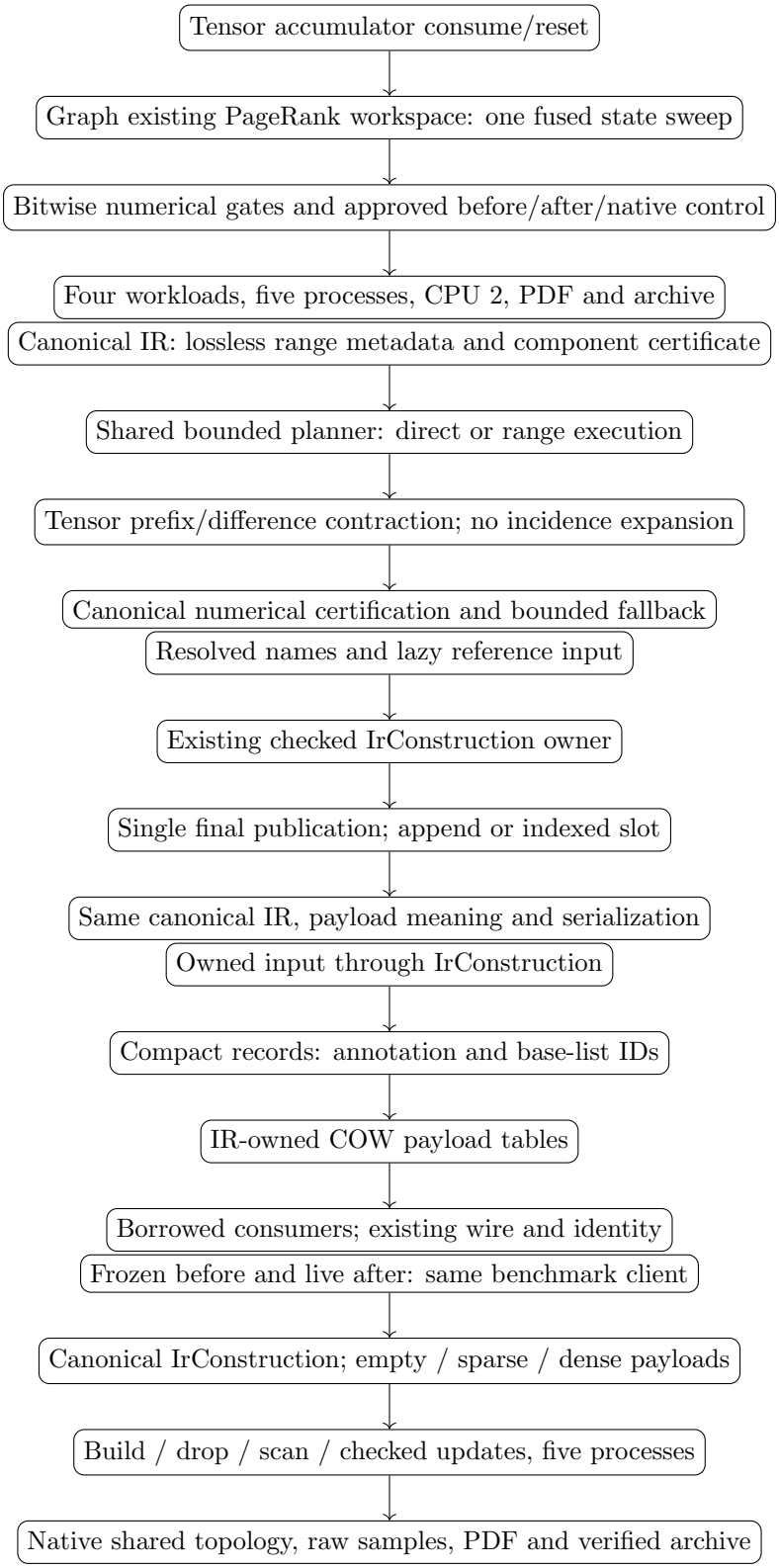
610 of 8070 compile samples to this function. Resolve the annotation once and clone its resolved value into the existing weights field; both owned wire fields must remain. Preserve list depth, absent versus empty, floating-point bits, symbol insertion order and first error. Extend existing test_ref_values.rs with mixed-value and error controls before editing production; extend the normalized-IR corpus for the real-parser path. Extend the existing Criterion generator with an explicit mixed-reference payload, retaining the numeric fixture and distinct benchmark identities. Archive matched binaries; compare lowering and cold compile in ABBA order, ten samples after warmup, with raw samples, IQR, worst and process peak. Existing 50k and 500k numeric fixture bounds apply; mixed 50k smoke must precede repeated runs. Each process capped at 180 seconds / 16 GiB. ETA approximately 20 minutes: tests and baseline 5, implementation and measured pairs 7, workspace gates and report/audit 8. No API or dependency changes. Bounded audit covers ModuleStore, intern/constant/reference resolution and lowering; record proven repeated work separately from unmeasured candidates and necessary semantic passes. Other candidates are not automatically bundled into this patch.

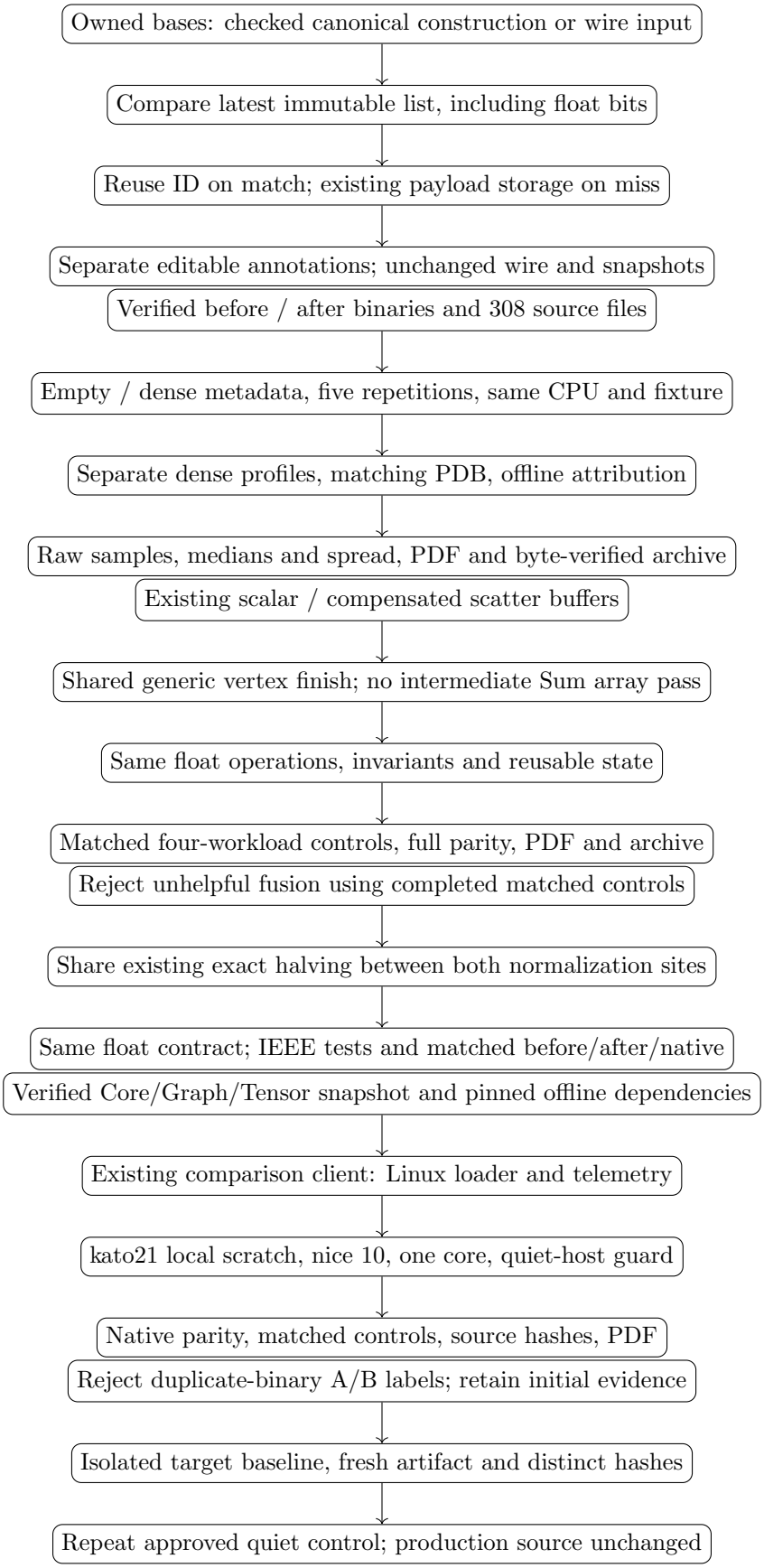


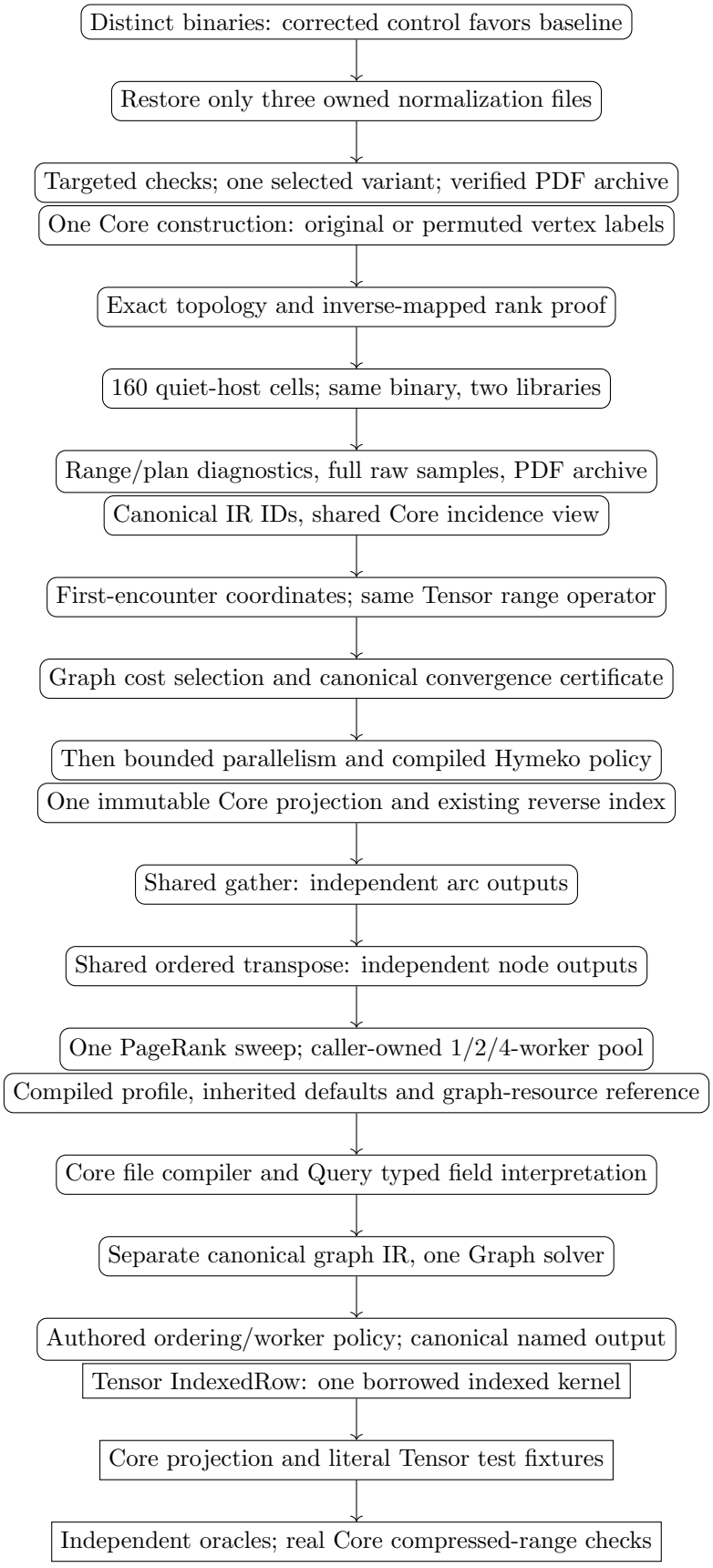

Configure and reserve update / symbol-occurrence capacities
Sort target positions; validate in original input order
Reuse staged values, changed IDs, symbol and hash deltas
Return snapshot or error; clear scratch contents
Changed old/new leaf values
Sort/combine symbol occurrences; exact cached counts
All live: shared metadata / dead symbols: existing compaction
Full coherence/model checks and incremental digest
Named parents: child counts
Prefix sums: row ends
Reverse scatter: child IDs and row starts
Existing bottom-up subtree fold
Owned and borrowed incidence API: one declaration template
IncidenceIndexData: one numerical owner
Four typed access families: shared declarations
Assembler apply: one resync cleanup boundary
Assembler cases, taxonomy and model tests
AssemblerRunner: Producer and TransactionAssembler
Real wire admission and common invariant checks
Eight builders share OperationFactory context and limits
Uniform upload, tensor binding and operand diagnostics
DeclarationFields: node/edge field contract
AstInterner mapping / ConstValues substitution
IncidenceCoefficients: live references / retained coefficients
IncidenceIndexData: one ordered transpose-row fold
Shared serial traversal / disjoint Rayon outputs

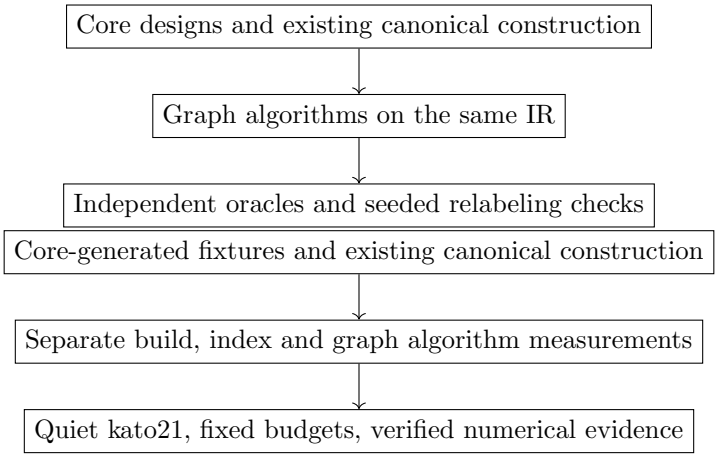

Shared declaration hashes in 256-entry pages
Checked values: detach only dirty hash pages
Same canonical encoder and complete fold
Identical harness and pinned CPU: before / after
Five repetitions per version and size
Raw samples + parity + bounded resources + PDF
Canonical byte ranges without a full concatenated buffer
Dirty intervals + shared BLAKE3 subtrees
OnceLock evaluation at the existing identity boundary
Input checks then Keep / Replace staging
Check every leaf binding; write replacements only
All-target symbol bookkeeping; actual-change hash delta
IrBuilder: lowering / synthetic / editor
Optional lazy load then IrConstruction + capacity hints
Compact references + IR-owned payload tables
Existing wire and digest contracts
Identical membership and mathematical contract
Existing native APIs and full result parity
Shared sampler: five processes, nine samples
Default-path audit and archived PDF evidence
Canonical Ir + existing incidence owner
Shared lazy projection: declaration / node domain
One gather / scatter implementation
Prepared signed operator ordinary sum
PageRank policies compensated sum


Shared canonical incidence projection
Exact unit counts + seeded ordered fold
Numeric parity + allocation gates
Existing selected comparison + PDF
Canonical incidence rows
Two-slot gather and sequential scatter
Same accumulator contract and result checks
Existing native comparison: 2 / 32 / 1000
Existing fixtures and canonical Core construction
Mixed widths / hub / components / chain / scales
Small correctness gates and frozen release binaries
User quiet-host signal, then existing measurement runners
Full parity, distributions, PDF and verified archive
Tensor: ordered sparse row contracts and contraction planner
Core: two borrowed IR projections and explicit alignment
Graph: shared PageRank contraction and existing Berge A*
Integer bounds prune; entropy orders; independent correctness gates
Existing contraction planner + serial CSR SpMM
Five-process runtime comparison; plan / prepare / execute / once
Canonical IR or graded cells -¿ shared Levi source / session / DSU
Incidence connectivity certificate + PDF + verified archive


Tensor accumulator consume/reset
Graph existing PageRank workspace: one fused state sweep
Bitwise numerical gates and approved before/after/native control
Four workloads, five processes, CPU 2, PDF and archive
Canonical IR: lossless range metadata and component certificate
Shared bounded planner: direct or range execution
Tensor prefix/difference contraction; no incidence expansion
Canonical numerical certification and bounded fallback
Resolved names and lazy reference input
Existing checked IrConstruction owner
Single final publication; append or indexed slot
Same canonical IR, payload meaning and serialization
Owned input through IrConstruction
Compact records: annotation and base-list IDs
IR-owned COW payload tables
Borrowed consumers; existing wire and identity
Frozen before and live after: same benchmark client
Canonical IrConstruction; empty / sparse / dense payloads
Build / drop / scan / checked updates, five processes
Native shared topology, raw samples, PDF and verified archive


Owned bases: checked canonical construction or wire input
Compare latest immutable list, including float bits
Reuse ID on match; existing payload storage on miss
Separate editable annotations; unchanged wire and snapshots
Verified before / after binaries and 308 source files
Empty / dense metadata, five repetitions, same CPU and fixture
Separate dense profiles, matching PDB, offline attribution
Raw samples, medians and spread, PDF and byte-verified archive
Existing scalar / compensated scatter buffers
Shared generic vertex finish; no intermediate Sum array pass
Same float operations, invariants and reusable state
Matched four-workload controls, full parity, PDF and archive
Reject unhelpful fusion using completed matched controls
Share existing exact halving between both normalization sites
Same float contract; IEEE tests and matched before/after/native
Verified Core/Graph/Tensor snapshot and pinned offline dependencies
Existing comparison client: Linux loader and telemetry
kato21 local scratch, nice 10, one core, quiet-host guard
Native parity, matched controls, source hashes, PDF
Reject duplicate-binary A/B labels; retain initial evidence
Isolated target baseline, fresh artifact and distinct hashes
Repeat approved quiet control; production source unchanged


Distinct binaries: corrected control favors baseline
Restore only three owned normalization files
Targeted checks; one selected variant; verified PDF archive
One Core construction: original or permuted vertex labels
Exact topology and inverse-mapped rank proof
160 quiet-host cells; same binary, two libraries
Range/plan diagnostics, full raw samples, PDF archive
Canonical IR IDs, shared Core incidence view
First-encounter coordinates; same Tensor range operator
Graph cost selection and canonical convergence certificate
Then bounded parallelism and compiled Hymeko policy
One immutable Core projection and existing reverse index
Shared gather: independent arc outputs
Shared ordered transpose: independent node outputs
One PageRank sweep; caller-owned 1/2/4-worker pool
Compiled profile, inherited defaults and graph-resource reference
Core file compiler and Query typed field interpretation
Separate canonical graph IR, one Graph solver
Authored ordering/worker policy; canonical named output
Tensor IndexedRow: one borrowed indexed kernel
Core projection and literal Tensor test fixtures
Independent oracles; real Core compressed-range checks



PERF-1 integration, 2026-09-09

Created-at: 2026-09-09T18:54:44+02:00. ETA: approximately 60–90 minutes, based on cached builds, existing 1-minute Core tests, 1–3 minute paired benchmark cells, SDK rebuild and browser tests; no complete historical campaign rerun. User approved saving the four-step strategy and executing step 1 in this turn. Existing authorization: `APPROVED-CORE-EDIT: compiler-repeated-work`; the course-module-compilation approval was the user's affirmative reply to that named scope, not a literal token. No new semantic or accelerator work.

Reuse the released course branches (Parser a528d98, Core 8623eb1) as bases. The original dirty performance checkouts remain the preserved source of the measured edits; independent integration worktrees hold the released control and candidate during A/B builds. Core main's independent sparse commit is not silently merged. Retire the temporary worktrees after evidence and publication.

Integrate only the existing grammar/string-scan optimizations, owned-record benchmark adapter, root borrowing/shared canonical paths, compact Pruefer state and single annotation-value resolution, plus their existing tests. Keep final course tests and diagnostics from the published branches. Affected additional paths: Parser grammar/lexer, existing benches/formats, tests/format_comparison.rs, scripts/format_*.py and their tests, Parser Cargo files and existing plan; Core canonical_hash.rs, module_store.rs, resolve.rs, existing interning benchmark and frozen/ref/module-store tests, Cargo pin/lock, generated SYMBOLS.md and docs/hymeko-platform-strategy.md; SDK Cargo pin/lock, existing changelog/package README and generated package. Root report exclusions remain. No Graph/Transformation/Tensor source change.

Use identical harnesses to build released control and integrated candidate. Run large-case smoke before bounded measurements: 50k linked/long-text parser and 50k mixed Core; 500k Core correctness/resource smoke where affordable. Minimum five post-warmup samples, retain raw values, median/IQR/worst and peak memory, binary/source/fixture hashes, affinity and environment. Reuse existing Campaign/ProcessRunner, Criterion and plotting tools. A/A or repeated control bounds ambient noise. More than ten percent stable regression requires investigation; inconclusive is not pass. Existing profiles justify the already-measured optimization mechanisms, not a fabricated combined speedup. No tests/builds overlap timed cells. Max process peak 16 GiB, timed process 180 seconds. Preserve full-IR and corpus hashes, signed-reference diagnostics, source provenance; run Parser, Core, native/Node/browser/Worker parity and actual packed SDK installation. CPU tests only; no GPU performance claim.

Publication: provider before consumer, immutable Git revisions, one Core identity, no local path patches shipped; verify GitHub and LAN refs. Reuse current course branch without default-branch merge. Archive reports to Samba. Rollback uses previous published SHAs and retained original SDK; never reset or remove unrelated user work. PERF-2/3/4 remain planned at task completion.

Current full-IR characterization, 2026-09-13

Created-at: 2026-09-13 15:37 +02:00. ETA: about 15 minutes for existing target builds, correctness, bounded cells and report. User explicitly requests the full IR benchmark and the compiled library size. No circular-buffer remeasurement, profiler, production change or second compilation pipeline. Reuse ModuleStore::compile through the existing interning benchmark: 50k numeric and mixed sources, cold compile and cache hit reported separately. Reuse existing canonical-CBOR corpus encode/decode cells as independent boundary measurements; do not add phase medians to claim an end-to-end result. Ten or more post-warmup Criterion samples, median/IQR/worst of per-sample iteration averages, no per-call p99 claim. Run the exact public-facade corpus round-trip correctness test first. Use existing measure_peak_memory.ps1 Program mode with explicitly built targets, at most 16 GiB observed working set and 180 seconds per process. A single same-scale smoke precedes repeated cells. Build/check work never overlaps timing. Fresh release library build: default-empty features, no IPC/Arrow/GPU; report rlib versus cdylib/PDB and inspect DLL exports. A small DLL without callable Core exports is not the full linked Rust dependency footprint. Affected tracked files: this existing plan.tex and rebuilt plan.pdf only. Append results to reports/2026-09-07-compiler-repeated-work.md and reuse its compiler-repeated-work evidence directory for raw samples, resource records and plots. Preserve the other session's uncommitted circular-buffer bench, manifest and plan changes. Rollback: revert this plan addendum; no runtime migration.

Post-extraction audit and performance, 2026-09-23

Created-at: 2026-09-23 23:10 +02:00. ETA: 10–15 minutes including release builds. User explicitly requested duplication inspection followed by performance benchmarks. Reuse current dirty Core, Tensor and Nagare checkouts; no commits, worktrees, source backups or production optimizations. Fetch succeeded in all three. Pin the exact source hashes in the measurement record.

Audit the combined production sources of Core, WGPU, Tensor's numeric crate and Nagare HNN with the existing token detector, using the same scope for Git HEAD and current files. Report relocation separately from removed duplicates. Inspect the surviving clone groups and distinguish shared contract duplication from CPU/GPU strategies and different mathematical operators.

Artifact ledger: extend the existing interning benchmark with a semantic-digest characterization group (full digest, cached read, checked leaf update including clone/validation); reuse its deterministic source generator and public Core API. Verify equality against the full digest before timing. Reuse the existing CSR benchmark without copying its kernels or fixtures. Keep measurement opt-in. Write reports/2026-09-23-core-tensor-audit-performance.md and raw JSON/logs/plots under the existing reports/compiler-repeated-work directory, prefix cutover-. No new runner or plan directory. Existing peak-memory runner and Criterion own execution and sampling; no build/test overlaps measured cells.

Measure 50k-node cold/cached compilation and small/medium CSR SpMV/SpMM at 24 Rayon threads. Semantic digest uses a bounded 1000-node fixture with one editable value. Minimum 10 warmed-up samples; median, inclusive IQR and worst sample average (not per-call tail latency). Use historical same-host samples only where source/config match; absence is a new baseline, not evidence of speedup. Report OS, CPU, Rust version, flags, fixture digest and process peak working set. Userspace RSS guard: 16 GiB; stop an individual timed process at 180 seconds. A greater than 10 percent change is investigated before a causal claim. Retain raw results even if the host is noisy. CPU results only; no GPU/remote claims. Archive the report using the existing Samba synchronization tool. Rollback of benchmark-only changes uses Git; generated measurement evidence stays in reports.

Canonical incidence owner, 2026-09-24

Created-at: 2026-09-24 00:23 +02:00. ETA: 8–12 minutes for the focused refactor, cached targeted tests and report. User explicitly requested a struct/trait-oriented incidence API with hash() and bytes(); this authorizes the Core canonical hashing edit. No benchmark or dependency change.

Extend canonical_hash.rs with CanonicalIncidence, borrowing immutable IR, and a private output trait implemented by Vec¡u8¿ and the existing BLAKE3 hasher. One record iterator and encoder serve both methods. Normal hashing streams ordered edges. Duplicate paths or wrapped u16 positions require sorting lightweight records, using the same encoder without a byte arena or fallback pipeline. Preserve the existing byte order, domains and hash config. The existing public program-hash facade delegates to the owner; no old incidence free functions or forwarding aliases remain.

Affected files: canonical_hash.rs, existing frozen_hash.rs tests, generated SYMBOLS.md, this four-format plan and the existing Core/Tensor audit report. Fresh fetch of all Core remotes succeeded; HEAD 10138bf equals the task upstream and is three commits ahead of origin/main. Preserve all prior dirty work; no new checkout, backup, commit or publication.

Verification: fixed payload bytes and header-plus-payload digest equality; empty incidence, mixed signs, repeated targets, edge ordering, duplicate edge paths, u16 arc/slot wrapping and long path truncation. Existing frozen whole program vectors and generated forest/config corpus remain unchanged. Run only the affected correctness target, library lint and touched-file format checks. Worst input has many incidences with colliding paths: sorting holds record descriptors, never duplicate encoded payloads. No speed or memory improvement claim without measurement. Rollback is the scoped Git diff; no source backup. Archive the amended report through the existing shared report-sync tool.

Pruefer owner, 2026-09-24

Created-at: 2026-09-24 00:47 +02:00. ETA: 8–12 minutes for a focused source refactor, cached correctness checks and report. The user explicitly requests struct/trait orientation for Pruefer, authorizing the Core edit. Core fetch succeeded on origin and github; task HEAD/upstream remains 10138bf, three commits ahead of origin/main. Preserve existing uncommitted work.

Replace the tuple forest and free Pruefer orchestration with SkeletonForest methods, a prepared CanonicalPrufer owner, and PruferTraversal implementing Iterator. Store degree, neighbor XOR, live vertices and leaf heap in the traversal. Share the existing CanonicalSink trait for byte/hash output; one token iterator determines both. The canonical program hash uses this owner directly. No compatibility wrappers or duplicate algorithm remain. Preserve leaf priority (subtree hash, path tie, vertex), super-root handling, iterative subtree folding, token order, headers and malformed-input behavior.

Affected files: canonical_hash.rs, existing frozen_hash.rs tests, generated SYMBOLS.md, this four-format plan and the current Core/Tensor audit report. Update stale helper-name comments in coherence.rs and the existing hierarchy and test-oracle files; these edits do not change executable behavior. Extend fixed hash vectors with public byte/hash parity, empty/single/multiple roots, interleaved anonymous declarations and generated forest shapes/configs. Exercise invalid cycle behavior explicitly. Run the frozen-hash and existing ordinary hierarchy correctness cases, excluding the ignored resource probe; library Clippy and touched-file formatting. No benchmark or dependency change. Worst-case deep trees remain heap-traversed; hashing streams fixed-size tokens, without constructing the full Pruefer payload. No measured speedup is claimed. Rollback is the scoped Git diff, preserving the preceding incidence change. No new checkout, source backup, commit or publication. Archive the final report.

Remaining canonical helpers, 2026-09-24

Created-at: 2026-09-24 00:57 +02:00. ETA: 6–10 minutes for the Core boundary; a public API migration may additionally require downstream checks. User requests remaining free functions to move into their responsible structs. Core fetched both remotes successfully; existing HEAD/upstream 10138bf remains.

CanonHashCfg owns domain-header initialization and chunk hashing. CanonicalPaths owns construction from IR, reverse paths for resolution-boundary validation and resolved strings. SkeletonForest owns its node-domain mapping. Reuse these owners from Pruefer, incidence, path-index construction and program hashing; remove their free helper bodies. No new struct solely to hold static utilities. The user explicitly approved public entrypoint removal with consumer migration. Promote CanonicalPaths::new/hash/validate; add DerivedPathIndex::new. Delete the three free public entrypoints and migrate Core, web, XR, Kawamirai, Transformation, Graph, the C++ Rust oracle and Nagare call sites. Keep manifest and lockfile pins unchanged; local verification uses command-line path patches. DerivedPathIndex is the existing owner for path-index construction. The XR check also reaches the existing simulator-xr-publish checkout. Migrate its callers and the primary simulator, Python binding and robot middleware found by the workspace-wide search. Exclude inactive alternative checkouts and the retired monorepo. Python's section-name adapter additionally requires the already-existing CanonicalIrCbor enum case to compile against current Core.

Discovery ledger: find canonical_hash.rs and existing consumer Rust files; grep the three public names and all private helpers across framework/Nagare; symbols confirm CanonHashCfg, CanonicalPaths, SkeletonForest and DerivedPathIndex already own these concerns; read implementations and callers. Verdict: EXTEND existing owners, remove free bodies, migrate callers in place.

Affected Core files: canonical_hash.rs and focused frozen/path/namespace tests; the existing plan bundle, symbol index and audit report. If public entrypoints are migrated, their Core consumers and downstream callers are also in scope. Keep canonical bytes, error variants and iteration behavior unchanged. Verify fixed hash vectors, byte/hash parity, namespace failures, duplicate paths and normal lookup behavior. No benchmark or dependency upgrade. Preserve prior edits; rollback is only this continuation's diff. No new checkout or source backup.

Namespace validator owner, 2026-09-24

Created-at: 2026-09-24 01:15 +02:00. ETA: 3–5 minutes. User requests a dedicated validator struct. Core fetch succeeded on both remotes; task HEAD 10138bf equals upstream and remains three commits ahead of main. Preserve the existing checkout and all preceding changes.

Discovery ledger: search IR/resolution for Validator and Validation owners finds none; CanonicalPaths in SYMBOLS.md and its implementation own the present validation. Read the method, namespace errors and all validation call sites. Verdict: EXTEND canonical_hash.rs with NamespaceValidator borrowing an existing CanonicalPaths, and MOVE validation plus reverse-index preparation into it. Remove CanonicalPaths::validate, with no forwarding method or free wrapper. Keep namespace errors and canonical hash bytes unchanged. Migrate lowering, CBOR validation and existing tests/bench call sites; retain shared path preparation. Run namespace/frozen-vector and lowering-boundary correctness tests, CBOR tests, compile the affected bench and run library lint. Update symbols and duplication gate, append the existing report and archive. No measurements or dependency edits. Rollback is the scoped diff; no backup or new checkout.

Subtree fold optimization, 2026-09-24

Created-at: 2026-09-24 01:21 +02:00. ETA: 4–6 minutes. User requests a more efficient subtree-hash implementation. Both Core remotes fetched; HEAD 10138bf still equals task upstream. Preserve current checkout and prior edits.

Discovery: SkeletonForest::new is the sole owner of parent/children/root construction; subtree_hashes is its sole fold. Each named declaration is inserted once under at most one valid named parent, or into roots. Therefore a root-reachable cycle or shared child is impossible even for malformed input; cycles and their descendants are disconnected from roots and retain zero hashes. Verdict: EXTEND the existing method, no additional module or production owner.

Build one root-first order, fold it backwards, and write directly into a dense zero-initialized HashId vector. Remove Enter/Exit frames, per-node Option state, on-stack flags and final conversion. Reuse one child-hash scratch buffer sized to maximum reachable fanout; skip it for leaves and single-child nodes. Use in-place unstable lexicographic sort: equal hashes contain identical bytes, so stability has no semantic effect. Cache the two configured named-leaf hashes within the fold, never across calls or configurations. Preserve all hash bytes and disconnected-component behavior. Complexity remains linear traversal plus sum of child-sort costs; no measured speedup is claimed.

Verify existing frozen vectors/configuration corpus, dense-ID permutations, deep/wide trees, mixed node/edge leaves, incidence multiplicity and malformed cycles. Extend the existing cycle regression to include a rooted component and a descendant of the disconnected cycle. Run only these correctness tests and library Clippy; no benchmark or profile was explicitly requested. Update the existing report, diagrams and symbol/duplication gates, then archive evidence. Rollback is the scoped diff; no backup, new checkout or dependency change.

Current Core performance, 2026-09-24

Created-at: 2026-09-24 01:30 +02:00. ETA: 3–5 minutes. The user explicitly asks about performance benchmarks. Measure the existing interning Criterion harness only: 50k canonical hash, cold/cached compile, and 1k full/cached digest plus checked leaf update. Reuse the existing memory runner and raw-sample summary machinery; no new benchmark implementation, source backup or worktree. Both Core remotes fetched, HEAD 10138bf equals the task upstream.

Discovery/cached evidence: the September 23 report has paired compiler/digest results. Its executable still exists and SHA256 matches 84f5f2261a8db3946b3d- afa17fd789ba589c038006d40f0ce7ec1d16c5de Run it twice before rebuilding, then the current source twice, with explicit –bench, numeric 50k fixture, 24 Rayon threads, 1-second warmup and existing 10/20-sample groups. This compares the accumulated canonical refactors, not the isolated last subtree change. Verify source and canonical fixture hashes across binaries. Build time is excluded.

Budgets: each measured process below 16 GiB observed peak working set and 60 seconds wall time, expectation approximately 30 seconds from the prior 24.8-second five-cell run plus the hash cell. Preserve raw samples, report median/IQR/worst sample average and two-run drift. Memory polling uses the existing 100ms userspace cap; record its limits. Run sequentially without builds or tests during timing. The Windows/275HX interactive host has background GUI activity (largest observed process 0.156 CPU seconds per second); do not claim a quiet-host release gate or attribute every difference to the subtree fold. Investigate differences exceeding 10 percent within the measurement scope; profiling remains unrequested, and causal speedup claims require separate evidence. Keep source/manifest/executable provenance, amend this report and archive results. No changes to production source, dependencies, public APIs or published pins.

Canonical hash batching, 2026-09-24

Created-at: 2026-09-24 01:45 +02:00. ETA: 6–10 minutes including archive. Continuation of the requested Core performance measurements and optimization. Owner: Core canonical hash; the existing user authorization covers this private implementation. Both remotes fetched; HEAD equals task upstream, three commits ahead of main. Preserve unrelated dirty work and all public contracts.

Discovery ledger: CanonicalSink is the sole output strategy in canonical_hash.rs; no buffered hash sink exists in Core. Extend that strategy with one private CanonicalHasher (configuration/header, fixed 16 KiB buffer, write, finish). Replace the direct Hasher sink, not the encoder. Existing frozen_hash tests, interning_bench, memory runner, report and plan bundle are reused. New measurement artifacts belong to reports/compiler-repeated-work with batching prefixes.

Evidence: the existing resolved Pruefer profile includes 463 Hasher::update samples, 255 ChunkState::update samples, and 371 path-string samples. These are historical inclusive observations, not current causal attribution. Current source issues per-field 1–32-byte updates. Test the bounded batching hypothesis against the just-measured current binary (canonical hash 78.743 ms); do not claim it explains the historical 6.6 percent regression without isolation. Batch the 24-byte header in CanonHashCfg and pass the existing sorted child-hash array as one contiguous slice. No alternate pipeline, dependencies or hash-version change. Fixed buffer memory is O(1), independent of model size.

Acceptance: identical frozen hashes, configured headers, long UTF-8 prefixes, wrapped incidence slots, empty streams, exact/full/partial buffer boundaries, and repeated calls. Existing targeted correctness gate and library Clippy. Two before and two after runs of only 50k canonical hash and cold compilation, existing warmup/sample counts, sequentially, no concurrent build or test. Expected 12–15 seconds per process, 40-second budget, 16 GiB observed peak cap. Record executable hashes, raw medians/IQR/worst, peak working set and fixture parity; interactive host remains a limitation. No fresh profiler run: reuse existing attribution and measure the scoped change. Retain only if parity passes and paired timings improve beyond ordinary run variation. Report and archive.

Remaining local duplication, 2026-09-24

Created-at: 2026-09-24 01:58 +02:00. ETA: 4–6 minutes including archive. User explicitly approved both identified cleanups and a fresh duplication audit. Owner: Core. Existing task checkout fetched both remotes; no upstream drift. Remove ir/common.rs and its module declaration; migrate the two integration-test consumers to SignedRefR::target/sign. Search found no external use in the framework/Nagare source trees. Public removal is intentional; no compatibility facade, replacement module, dependency change or extra checkout.

Discovery: TransactionAssembler already owns abandon and the seven staging-limit guards. Add one private staging_limit_exceeded method there; preserve guard order, counter updates, all diagnostic strings, committed version and replay position. Strengthen existing byte-limit, chunk-limit and superseded-drain tests to verify cleanup and preserved committed/sequence state. Run the assembler unit test module, the two migrated integration-test modules, library Clippy and fmt. No benchmark or profiler is requested for this cleanup.

Compare the same token detector and scope before/after: entry 462 marked runs, 9588 marked lines, 185 scanned files. Separate source and test contributions; inspect the remaining largest Core matches before calling any a duplicate pipeline. Existing candidate inventory, report, symbol index and diagrams are updated in place; report archived to the existing Samba destination.

Shared little-endian reader, 2026-09-24

Created-at: 2026-09-24 02:12 +02:00. ETA: 5–7 minutes including archive. User explicitly requests struct/trait/generic extraction of envelope readers. Owner: Core ir_stream. Both remotes fetched; existing task checkout is current. Discovery found no shared endian reader in common or ir_stream. New private reader.rs owns LittleEndianReader over a borrowed byte slice and a LittleEndianValue trait with associated fixed byte array. Generic read performs one shared bounds-checked copy, then statically dispatches the type conversion. A compact macro supplies primitive conversions for the four actual consumers: u16, u32, u64, f32. SnapshotVersion implements the same private trait by reusing its existing public conversion. No external reader API or dependency.

Migrate envelope fields, section-table fields, typed section-body copies and snapshot counters. Remove the old rd functions and per-version closure. Keep length/resource checks before reads/allocations, preserve exact validation order and wire errors; a private reader called outside validated bounds may panic, while public decoding must reject every truncated input. No pointer casts, unsafe code, dynamic dispatch or parallel decoding implementation.

Tests: independent known bytes at unaligned offsets and exact buffer ends; integer maxima, floating-point bit patterns, invalid offsets/short input; existing envelope/section/version/codec unit tests and stream-admission integration tests. Library Clippy, fmt, symbol-index and same-scope duplication checks. No performance measurement requested. Update existing candidate inventory/report, archive evidence.

Incidence and access surfaces; assembler cleanup, 2026-09-24

Created-at: 2026-09-24 02:39 +02:00. ETA: 6–8 minutes including archive. Owner: Core. User approved incidence consolidation and further access/assembler inspection. Both remotes fetched; existing task branch has no upstream drift. Discovery finds one numeric incidence implementation already in IncidenceIndexData. Share the 14 inherent delegating methods with a private declaration template, parameterized by data lifetime: owned results borrow self, view results retain the original data lifetime. Preserve Copy/two-pointer view layout, inherent call sites, operator arithmetic/order and all public signatures. A trait-only replacement would require consumer imports; a naive generic borrow shortens view lifetimes. No new runtime abstraction, dependency, module or compatibility API.

In access.rs share the four typed iteration/count/containment declaration families. Keep existing typed lookups and node/edge backlink checks, including the arc exception. Templates describe a repeated public surface; they do not create a second algorithm. Centralize transaction abandonment at public apply on every ResyncRequired outcome; retain diagnostic state captured before cleanup and replay bookkeeping rejection. Reset also reuses the existing abandon operation.

Extend existing incidence parity test with results surviving a short-lived view (slice, iterator, data reference and operator plan). Existing assembler test driver checks every resync preserves committed version/digest/sequence while clearing staged/draining state. Run focused incidence/access integration tests and assembler unit tests, library Clippy, formatting, symbol and duplication gates. No benchmark. Compare the same detector against 461 marked runs / 9546 marked lines / 185 files; separate declarations and test fixtures from actual pipeline duplication. Update existing inventory and report; archive to Samba.

Assembler test runner, 2026-09-24

Created-at: 2026-09-24 02:55 +02:00. ETA: 6–8 minutes: one test-only source refactor, targeted test compilation and archive. User requests runner structs for test duplication too. Owner: Core

ir_stream tests. Fresh fetch of both remotes; upstream 0/0. Existing dirty checkout and all production behavior preserved.

Extend existing tests_support with AssemblerRunner owning TransactionAssembler and the existing Producer. Move the free apply helper and its real wire validation and resync invariants into the runner. Reuse that path for step, expected outcome and ordinary complete-transaction execution. Migrate unit tests, ignored taxonomy probe and model tests; keep explicit bytes for exact/conflicting replay and expose producer controls for sequence gaps, stream changes and exhaustion. No alternate validator, test-only acceptance bypass, dependency or public production API.

Retain scenario-specific assertions and reference-model independence. Remove one byte-identical duplicate replay test only after confirming the surviving test covers all its assertions. Consolidate send/apply setup where bytes are used once; keep fixtures reused by replay tests. Preserve raw message fields/order for every migrated scenario. Existing correctness tests themselves prove runner behavior.

Affected: assembler.rs test modules, existing plan bundle, candidate inventory, report and generated symbol index if needed. Source is CORE-protected; the user's current runner request authorizes this test-only change. No production sections will change. Run the assembler unit and model-test modules (taxonomy stays ignored), Clippy for the library test target, rustfmt/diff checks, symbol and same-scope duplication gates. No performance measurement. Before: 459 marked runs / 9279 marked lines, assembler 663 of which 607 in test code. A smaller source count must not be bought by weakening tests or moving identical bodies into another file. Worst cases retained: u64 sequence boundary, zero replay capacity, staged/drain limits, randomized and exhaustive message streams. Restore only this task's patch through Git if needed; no backup checkout. Append report and archive to Samba.

WGPU preparation and declaration traversal, 2026-09-24

Created-at: 2026-09-24 03:08 +02:00. ETA: 10–15 minutes: eight backend builders, two AST passes, focused correctness and archive. User explicitly approved WGPU and traversal duplicate removal. Owner: Core checkout (hymeko_wgpu backend and hymeko_core resolution). Fresh origin/github fetch; upstream 0/0; reuse checkout. Existing roadmap invariants apply: canonical IR and numerical semantics unchanged.

WGPU: OperationFactory becomes the shared borrowed context for every builder, with one OperationLimits value replacing eight per-operation limit structs. Move typed uniform allocation/upload into that existing owner. GpuTensor owns its binding-entry construction. A private OperandCheck owns the operand diagnostic label and common required-value/shape-error construction. Remove six copies of each helper and the rotor-local uniform builder. Preserve each prepare method's validation order, diagnostics, kernel/layout binding positions, workgroup policy, resource usage flags, typestate and operand lifetimes. No new preparation-time collection, extra buffer, per-encode allocation, shader or numeric change.

AST: private DeclarationFields trait in existing sym_ast.rs exposes borrowed name/bases/optional children and mutable bases/children for NodeInner/EdgeInner. An AstInterner struct owns the interner borrow; one generic declaration method visits annotation/name/bases/body in existing order, with node/edge constructors preserving None versus Some(empty). ConstValues owns one generic mutable declaration walk. Arc ref-before-annotation substitution order remains explicit; error short-circuiting and constant arithmetic stay unchanged. Remove replaced free lowering/substitution helpers. Public entry points and parser dependency remain stable; no parser checkout modification or new module.

Files: eight operations/*.rs builders, operations/mod.rs, resources.rs; Core sym_ast.rs, resolution/intern_pass.rs and const_resolve.rs; focused tests in those owners as needed; existing plan/report/candidate/symbol artifacts. These source paths are CORE-protected; the current

user request authorizes them. No dependency, branch, worktree, commit or publication change. Rollback only this task's edits, preserving earlier dirty work.

Verification: Core intern/constant unit tests plus frozen identity/namespace integration fixtures; strengthen order/optional-body/error precedence checks. WGPU existing HSiKAN, entropy pooling, sparse scatter/highway/resident chain, rotor attention/spike tests plus targeted SpMM/activation preparation and dispatch cases. Inspect runners first; no profiling/benchmark opt-in. Execute GPU tests serially and require actual adapter execution when available. Clippy for both libraries, fmt/diff/symbol and duplicate gates. Before: 448 runs / 8996 marked lines / 185 files. No performance claim; shared buffers retain existing allocation sizes and ownership. Report actual results and any adapter skips; archive to Samba.

Incidence transpose row kernel, 2026-09-24

Created-at: 2026-09-24 03:24 +02:00. ETA: 6–8 minutes, one production owner, existing correctness fixtures, report and archive. User explicitly selected the remaining index/prepared-plan calculation overlap. Core-protected source changes are authorized by that request. Fresh origin/github fetch succeeded; upstream 0/0, existing dirty checkout retained. No dependency or public API change.

Introduce a private IncidenceCoefficients trait implemented by existing IncidenceView and IncidenceOperatorPlan. The live strategy resolves the canonical arc/ref coordinate; the prepared strategy reads its retained f64 coefficient. IncidenceIndexData owns one ordered transpose-row fold, shared by direct serial, prepared serial and Rayon applies. Share serial output traversal as well; preserve its initial zeroing, partial-write behavior and untouched output tail. Rayon keeps disjoint output partitioning and initial slice validation. Reuse the live coefficient strategy during preparation. Preserve coefficient multiplication and summation order, duplicate incidences, neutral references, signed zero and nonfinite policy. The direct path propagates nonfinite values; preparation rejects the first offending node in canonical node order. No dynamic dispatch, new retained array, per-apply allocation, IR copy, matrix materialization, reordered arithmetic or automatic cache. Bx and sparse topology construction remain in their existing owners.

Affected: incidence.rs, incidence_precompute_profile.rs correctness section, existing plan bundle, candidate inventory, symbol index if changed and report. Private trait and generic methods need no consumer migration. Restore only this task's patch if necessary, retaining earlier uncommitted work; no backup checkout.

Tests: reuse index/view/prepared and independent oracle fixtures. Add a compact runner covering cancellation-sensitive row order, duplicates, isolated nodes, empty graph/arc, repeat apply, output tails, overflow/neutral nonfinite coefficients and first-error node. Check bit parity including serial/Rayon; floating exceptional results by classification where NaN payloads are unspecified. Inspect aggregate selection first and keep ignored profiles disabled. Run incidence unit tests and specific incidence correctness integration filters, Clippy, scoped fmt/diff, symbol and duplication gates. Entry: 449 runs / 8543 marked lines / 185 files. No performance experiment or speed claim: current request is consolidation. Plan compile before source edit; write report and archive to Samba at completion.

Profiling continuation, 2026-10-11 17:11 +02:00. The user explicitly requests Core performance profiling and a fastest-framework objective. Measure merged Core `b66cb4d` in the existing checkout; the source tree is clean before this documentation amendment. ETA: 15–20 minutes for the bounded initial characterization, including build, repeated cells, CPU capture and analysis. Reuse `interning_bench`, `ir_bench`, `scripts/dev/measure_peak_memory.ps1`, `source_manifest.ps1` and `criterion_report.ps1`. No new benchmark owner or dependency is needed.

Initial matrix and hypotheses. First measure the existing 50k frontend phases, full/cached semantic identity and a checked leaf update. Repeat the main cells in two separate processes. After a bounded smoke, measure the existing 500k frontend's cold compile and structural hash, keeping its distinct source identity. Cover true higher-arity structure with the existing IR hash cells (1k and 10k nodes, arity 4 and 32). The competing explanations are parsing/lowering, canonical encoding, and immutable-model copying/validation; attribute each using symbolized stacks before selecting an optimization. Phase timings are not additive.

Measurement controls. Criterion release with debug information, fixed fixture hashes, raw samples, median/IQR/worst batch mean, and two repetitions. Samply records separate 10-second CPU windows for cold compilation and semantic digest/update paths; profiler times are not benchmark times. The existing Windows monitor records peak working set and observes the 16 GiB cap. Keep individual benchmark invocations below 180 seconds; log real progress, executable hashes, commands, CPU/power context and host interference. OS profiler consent may require user interaction; report it if capture is blocked. No services or other workloads are stopped. Save a timestamped report and numerical/profile/plot artifacts under `reports/2026-10-11-core-profile/`; archive with the shared report tool.

Acceptance and limits. Existing Core correctness gates are green on this revision (178 library, 152 IR verification and 7 doctests; Clippy clean). Fixture parity assertions must pass in the benchmark too. This phase changes no production code, hashes, public API or wire encoding. A profile justifies a specific subsequent optimization; it does not establish a world ranking. Such a ranking needs equivalent semantics, workloads and a separately agreed competitor matrix. RL, renderer migration and unrelated numerical/GPU campaigns stay excluded.

Optimization continuation, 2026-10-11 18:13 +02:00. The user explicitly asks to proceed with optimization and to deliver PDF reports. The completed CPU profile identifies symbol visitation and copying as update costs. Core owns both changes: `module_store/execution/value_updates.rs` lazily copies the base interner only when an update interns new text; `resolution/symbols.rs` keeps its one typed walk but marks path/alias keys without rebuilding their hash maps. Full hierarchy and model validation stay enabled. No representation, public API, hash byte, error contract, snapshot isolation or symbol-lifetime change.

Extend the existing `benches/interning_bench.rs` with an explicit digest fixture size and checked text updates. Preserve the original production binary/PDB pair after building this same harness, then change production. Reuse the benchmark's existing parity assertions; add a map-stability regression to symbol-compaction tests and run it against the old implementation first. Existing mixed/nested-value, text lifetime, invalid-input, old-reader and full/incremental parity tests remain discriminating gates. Run the 1k and 50k update workloads in paired ABBA order, with raw samples and the same Windows peak-working-set monitor. Measure full/cached digest controls at 1k. No new benchmark owner or dependency. Preserve the 16 GiB / 180-second per-process budgets.

Expected implementation, tests and measurements: 20–30 minutes. The copy-elision must retain the original symbol IDs until the existing compactor rewrites them; text edits must still own their extended interner. Marking must validate every symbol while leaving map keys and allocations intact; dense rewriting still rebuilds maps. Reject a regression exceeding 10 percent and investigate its cause. Record the result in a PDF beside the existing profiling artifacts, not a new Markdown report. Rollback only the named optimization/test/benchmark hunks, preserving prior task work. The existing checkout is reused on `perf/checked-value-update-symbols`; no new worktree. No push or merge is performed without the requested notification.

Measured continuation, 2026-10-11 18:31 +02:00. The first ABBA sequence shows numeric-update gains but no consistent text-update gain. The same symbol visitor allocates a worklist

even for a numeric or DeclId reference scalar, neither of which contains a SymId. Return immediately for these two scalar roots; string/list traversal and all validation remain unchanged. Preserve the first measured candidate and use separately named final ABBA samples, against the same original binary and unchanged benchmark harness.

Second optimization, 2026-10-11 19:05 +02:00. The user authorizes the next measured Core optimization. The previous candidate's symbolized update profile attributes 30.5 percent of selected stacks to hierarchy coherence validation: copied path vectors, sorting and a second lookup table. Core's existing `ir/coherence.rs` owns this validator; `PathKey` already supports borrowed symbol-slice lookup. Borrow the existing paths and query the existing Index instead of allocating per-path vectors and rebuilding a HashMap. Keep deterministic declaration/path ordering with an unstable sort: Index keys are unique, so ties cannot occur. All checks and report limits remain active. Typed SymId paths also avoid the previous lossy u32 conversion; add a regression for distinct 64-bit symbols with identical low 32 bits, plus exact ordered/capped findings across reversed index insertion and nearest-prefix/empty-path cases. This intentionally corrects false prefix matches for malformed wide symbol IDs; ordinary graph findings, public API and hash/serialization algorithms are unchanged.

Extend the existing coherence integration suite. Reuse the preserved first-candidate EXE/PDB as baseline and the unchanged interning benchmark. Run bounded 1k/50k smoke, then numeric/text update ABBA comparisons and the 1k digest controls; 16 GiB cap, under 180 seconds per process. Capture the optimized CPU profile and preserve source manifests. Run Core library/unified-IR correctness, Clippy and downstream checks. Report this second step and cumulative gains separately in PDF. Estimated duration: 15–25 minutes including build, paired measurements and reporting. Reuse this dirty task checkout and branch; preserve prior changes. Rollback only this continuation's coherence/test hunks. No commit, push or merge is requested by this continuation.

Third continuation, 2026-10-11 19:26 +02:00. The user requested commit/push and continued optimization. The previous two steps are published as `8a7d7f5` on `perf/checked-value-update-symbol`. The latest profile attributes 37.4 percent to deep IR cloning, principally owned incidence and weight vectors. Sharing those public Vec fields requires a consumer API migration, which is not silently bundled into this step. The next compatible copy reduction targets Interner construction (23.4 percent of update samples).

Core's existing `resolution/interner.rs` owns a private reference-counted table and immutable strings. Existing public methods remain unchanged; private fork and retained-symbol operations share unchanged data. Insertion detaches a shared table. `resolution/symbols.rs` retains the one typed symbol walk and uses the Interner-owned dense ID mapping. `execution/value_updates.rs` forks the symbol table lazily when new text is introduced. No old table is retained by a compacted table with dead strings. This also removes the fabricated static string references from the old interner. Tests cover detached mutation, dense ID mapping, dropped dead-string ownership, retained readers and existing digest/CBOR parity. No dependency, public signature or canonical encoding change.

Reuse the preserved `after-coherence/` EXE/PDB as baseline and the unchanged Criterion harness. Measure 1k/50k numeric and text updates, 1k digest controls, and 50k intern/cold-compile controls in ABBA order after bounded smoke. All cells retain the 16 GiB and 180-second process bounds. Reject unexplained regressions over 10 percent, including the compiler and memory controls. Capture a symbolized CPU profile, run owning and downstream correctness, and deliver a separate PDF. Estimated work: 20–30 minutes, including construction-regression measurements. Rollback only this continuation's Interner/SymbolCompaction/update hunks; keep the published first two steps and their evidence. The same checkout is reused.

Shared IR continuation, 2026-10-11 20:47 +02:00. The user explicitly authorizes Core shared storage plus Graph and Transformation adaptation; other consumers follow only after Core. Preserve the pending shared Interner changes on the current branch (HEAD `8a7d7f5`); fresh origin fetch succeeded, branch upstream is equal, main is one ancestor commit behind. Graph `ed7d0cf` and Transformation `90d64a8` match their freshly fetched feature upstreams. Transformation main diverges by 7/1 (the other commit is a material-lobes merge); reuse its existing feature integration base without discarding that branch's work or silently merging main. Existing TeX byproducts in both consumer checkouts remain untouched.

Core owns `ir/table.rs`: one contiguous `IrTable<T>` over shared Vec storage, copied on mutable access. Existing read accessors and ordinary indexing, iteration, append and slice operations stay available. Explicit Vec assignments or typed clone results may require conversion. Ir clone shares all nine tables; a leaf edit copies the declaration table only. This is table-granular, not paged or constant-time leaf editing. Meta stays unchanged. Serialization delegates to the same vector sequence; existing manual wire admission remains authoritative.

`resolution/symbols.rs` uses one macro-defined field walk instantiated with shared or mutable borrows; marking cannot detach any Ir table. Dense rewriting may detach tables when symbol IDs actually move. All validations stay active. Other Core edits: `ir/mod.rs`, `ir/ir.rs`, direct Vec assignments in `module_store/module_store.rs`, existing update/symbol tests and any compiler- identified table-boundary callers. Update the generated symbol index. Graph's explicit owned arc clone uses an explicit deep copy; adapt further proven consumers only after Core gates pass. Temporary local patches are integration aids, not published dependency pins. Do not publish an uncommitted provider revision.

Before creation: table preflight found StringTable; rejected because it is an immutable string/Python snapshot, not an editable generic IR record table. The generational-arena artifact is a retired representation experiment; no code is copied from it. Reuse existing plans, benchmarks, fixtures and tests. Add tests for mutable access/iteration/owned iteration isolation, sequence wire parity, retained old snapshots and unchanged-table sharing, including read-only symbol marking and dense remapping. Run Core correctness/Clippy first, then Graph and Transformation workspaces with one local Core identity. Reuse after-sharing EXE/PDB baseline; smoke 1k/50k, ABBA update and cold-compile/intern controls, existing IR hash/traversal controls, separate CPU profile, 16 GiB/180-second process bounds. Preserve all raw evidence and deliver PDF. Estimate 45–75 minutes: storage/walk implementation, ownership tests, consumer compilation and paired measurements. Rollback only these storage/migration hunks, retaining the prior Interner step.

Construction-control correction, 2026-10-11 21:00 +02:00. The first Arc-backed table candidate keeps cold compilation within 0.2 percent but regresses direct IR construction by 25 percent; it is not accepted. Preserve its binary and results. Keep a uniquely owned Vec during construction/mutation, and convert it without copying records to shared storage at CompiledProgram and CompiledModel publication boundaries. A first write detaches shared storage once; later writes use the owned Vec directly. Standalone mutable tables clone deeply; published snapshots share. Both preserve independent Clone semantics. No unsafe shortcut or reference-count bypass. Recheck the same construction/read controls, then run the update matrix. The stored earlier `ir_bench` baseline is valid for these controls because the preceding Interner/coherence changes do not enter their timed construction, raw state fold or accessor traversal; input and benchmark source are unchanged. Full digest/compiler/update controls use after-sharing.

CPU closure, 2026-10-11 23:33 +02:00. Windows consent was granted and both delayed captures completed. The rejected Arc-table construction candidate contributes 6,273 selected construction samples; 26.7 percent include Arc mutable-access handling. The final shared-IR update capture contains 9,143 samples, 4,850 in the actual value-update function. All 813 sampled Rust addresses resolve against the preserved final EXE/PDB. The final binary and all 191 manifest

files still match the measured candidate; no production code or timing matrix is changed in this closure.

Compare actual copy stacks, not only a function name whose work moved: the union of whole-Ir clone and declaration/node/edge/arc Vec clone frames falls from 52.2 to 16.9 percent of selected update samples. Topology-table copying has no samples in the final numeric workload; declaration-table detachment remains. Inclusive shares overlap and are not additive phase times. Keep the measured accessor traversal tradeoff and the independently reproduced pre-existing Transformation fingerprint failure visible. Update the existing PDF/JSON and archive the new profiles, resolved symbols and figures. No new commit or push is authorized here.

Algorithms and storage, 2026-10-12 01:09 +02:00. The user requests algorithm and data-structure selection followed by implementation, including heap storage. Fresh Core origin fetch succeeded; HEAD and feature upstream remain at 8a7d7f5, one commit ahead of main. Reuse the checkout and retain the pending shared Interner and IR work. Core owns canonical skeleton preparation and update bookkeeping; Graph owns traversal queues/workspaces; Tensor already owns numerical buffer arenas. No sibling source changes, RL work or publication.

Implement stable counting/prefix-sum packing in the existing SkeletonForest: replace one child Vec per declaration with a flat child-ID buffer and N+1 offsets. Reverse stable scatter may reuse cumulative row ends as cursors, leaving row starts without an extra scratch array. Preparation is O(N+E), row lookup O(1), row iteration O(degree), and exactly two child-storage buffers replace the outer vector plus nonempty row buffers. Keep the existing priority heap, labels, bottom-up fold, hash bytes and topology preconditions. No public signature change. The generic numerical CSR owner is not an IR hierarchy owner; introduce no new generic CSR abstraction. Searches found only this private child-list owner.

Affected files: this plan bundle, CORE.YAML, SYMBOLS.md if generated output changes, and `hymeko_core/src/ir/canonical_hash.rs`, including private correctness tests. Add independent row-oracle checks for empty, chain, star, mixed named/anonymous, out-of-range-parent and cyclic inputs, and exhaustive small parent maps. Run focused tests, existing frozen-hash/corpus/semantic identity checks and owning library checks; run formatting, Clippy and symbol/duplication gates. Worst inputs include a chain with a heap buffer for every original nonleaf, a wide star and mostly anonymous declarations. Offsets cover the full declaration-ID domain.

Deliver a Hungarian PDF catalogue with primary references, applicability, worst-case versus amortized bounds, allocation/reclamation costs and implementation order. Investigate delta propagation, inverse symbol-use indices, sparse sets, indexed heaps, bounded pools, TLSF and snapshot reclamation. Deadline/hardware/OS, maximum graph size and retained-reader bounds are still unspecified; ask for them without blocking this deadline-independent storage work. No new timing campaign; report analytical storage bounds separately from measured performance. ETA about 25 minutes: implementation/tests 10, sourced catalogue and PDF verification 15. No numerical latency improvement or hard-real-time guarantee is asserted. The existing 16 GiB workspace resource cap remains. Roll back only this continuation's named hunks, preserving earlier changes. Archive the final PDF and verification evidence through the existing report-sync tool.

Packed-child correctness closure, 2026-10-12 01:16 +02:00. Implemented in the existing private owner. Two new tests compare against a direct parent-scan oracle: all 3,529 kind/parent combinations for zero through three declarations, plus 257-vertex chain, star and branching cases. Core library: 187 passed, 2 ignored. Unified IR: 181 passed, 7 ignored, including frozen hashes, Pruefer cycle rejection and corpus CBOR parity. Strict Clippy and changed-file formatting pass; symbol index is current. Duplication remains 448 clones / 8,529 lines, below the baseline 484 / 10,411. No benchmark or new speedup claim. Preflight's filename candidate `hymeko_wgpu/`

`tests/skeleton.rs` is a GPU backend test suite, not a named-declaration hierarchy representation; rejected. No new production type or file was introduced. The implementation and tests are covered by source manifest `322c53db7343f2572b5c7f5cb9a942fe710991bdee16e0ee843f8bc54e70838e`. The algorithm/heap/allocation catalogue is delivered as `reports/2026-10-12-algorithms-storage.pdf`; selected future candidates are explicitly distinguished from this implemented storage change.

Approved catalogue continuation, 2026-10-12 01:31 +02:00. The user authorizes every catalogue candidate. Implement in owner order, Core before consumers; retain conditional applicability (no arbitrary global allocator replacement or invented hard deadline). Fresh origin fetch: feature HEAD/upstream 8a7d7f5 equal, one ahead of main, existing checkout retained. No new publication instruction is inferred. Preserve pending shared storage and packed-child work.

First deliver shared CompiledProgram Index/import ownership through Arc, with deep detach only when symbols are renumbered. CompiledProgram public field types therefore change; borrowed access coerces as before, owned construction needs conversion. Core CBOR wire payload keeps its existing owned Index/Vec types. The workspace search found one external constructor in hymeko_web's WASM document owner; adapt only after Core gates, with a fresh fetch and local rules. Core direct consumers include pretty-print and namespace-fault tests.

Cache exact model-local symbol-use counts lazily in CompiledModel. Generalize the one macro-defined field walk to read visitors; use it for full marking, counting and value deltas. Collect signed symbol occurrences only from changed old/new leaf values, sort by symbol ID and combine each run before adjusting counts. Equal multisets keep the shared count vector. With no unused symbol, reuse Interner and metadata without a full mark/retention scan. If a symbol dies, reuse the existing dense retention/rewrite path and compact the count vector in the same order. Retain all coherence/model validation and final digest folding. Numeric updates avoid count-vector copying after lazy initialization. Text updates may still copy O(S) counts/interner data and globally remap IDs: do not claim O(k) for the whole operation. Sorting a delta of R occurrences costs O(R log R).

Touched: CORE.YAML; this plan bundle; SYMBOLS.md generated index; `resolution/symbols.rs`; `module_store/module_store.rs`; `module_store/execution.rs`; `module_store/execution/value_updates.rs`; `writers/cbor_writer/tests.rs`; `util/pretty_print.rs`; `tests/unified_ir/verification/namespace_faults.rs`; compiler-identified direct Core callers. Reuse existing tests and add multiset/count parity against the independent full mark/compaction path; retained readers, no-op, scalar, nested lists, shared text, dead suffix, dense remap, invalid symbols, and imports/aliases. Check pointer sharing and count state without timing assertions. Run targeted Core library/IR/Clippy gates, then affected consumers. PDF evidence and archive. ETA 35–50 minutes for this first dependency pair: implementation 15, correctness 10, integration/report 10–25. No profiling matrix in this step. Existing 16 GiB resource cap applies. Roll back only named continuation hunks; preserve earlier work.

Parent-chain algorithm, 2026-10-12 01:40 +02:00. The first pair passes 191 library and 181 unified-IR tests plus Clippy. Continue within the approved catalogue: the existing parent-acyclicity validator repeats each ancestor chain, O(N times depth), or O(N times max-walk) on cycles. Classify each functional-graph vertex once with unseen/active/completed state and unwind an explicit path stack. Completed outcomes retain distance to root or invalid parent, or cycle reachability. Emit diagnostics afterward in the original declaration order and with the same capped collector. O(N) parent classification, O(N) scratch; other validator work is unchanged. This is not validation caching: every current input is still checked. No identity-equality shortcut.

Keep legacy diagnostic walked values for finite limits: overlong/rootless walks report max-walk plus one, invalid parents report the earlier invalid distance. At usize::MAX, saturate that diagnostic bound and terminate cycles instead of overflowing or looping. Modify only `ir/coherence.rs` plus this plan/manifest. An independent bounded naive-walk oracle checks every parent map up to four vertices at limits 0, 1, N and 2N; include cycles, out-of-range parents and NONE. Test

usize::MAX cycle termination and existing full-validator faults. Add about 10 minutes implementation/correctness. Retain all previously stated gates.

Algorithm implementation closure, 2026-10-12 02:05 +02:00. Core: 193 library, 181 unified-IR and 7 doctests passed; 9 ignored; strict Clippy passes. All 1,441 parent maps through four vertices match the bounded naive diagnostic oracle at four walk limits and three report caps. Delta tests cover full counts/compaction parity, repeated symbols, aliases/facets/imports, suffix removal, remapping, retained readers and count errors. Source manifest (191 files): `e516c5e60c5d2f37f2b44d3f`

Graph advanced externally to f7ed8c2 during integration; retain it and its published 605bc8c arena-test fix. The separately planned indexed frontier passes 644 Graph tests (30 ignored), including the allocation and path oracles, plus strict Clippy. Transformation current feature 0c1a2c5 contains the external emitter fingerprint correction: 330 selected tests pass (5 ignored, workflow timing test explicitly filtered), and all workspace test targets compile. An earlier suite invocation also ran its non-ignored workflow timing test; no performance claim is drawn from that incidental measurement. Generated SDF blank lines restored only after checking exact provenance/content. Web 4bfe66a needs three ownership conversions; 42 native tests and the wasm32 target check pass. All temporary lockfile changes were restored after hash checks. Single local provider identities were verified; these remain local integration builds, not publication pins.

Core duplication remains 448/8,529 below 484/10,411. Graph's stock gate fails against 547/12,629; current HEAD and candidate both measure 577/13,157 using the same provider detector. The heap adds zero clones; do not raise the baseline or describe that gate as green. Current symbol indices and formatting pass. PDF and evidence: `reports/2026-10-12-incremental-bookkeeping.pdf`, with Graph's copy and source evidence under its indexed-frontier report. Other catalogue candidates remain authorized; deadline/platform/capacity requirements were requested for the memory/publication work. No commit, push or merge by this continuation. Existing uncommitted work and user TeX outputs retained.

Commit and requirements clarification, 2026-10-12 02:14 +02:00. The user requests committing the completed changes. Prepare local commits in provider order (Core, Graph, Web); no new push or merge is requested. Existing test/source manifests identify the reviewed candidate. The user clarifies that hardware, deadlines, graph/delta limits and retained-reader limits are not fixed at this early framework stage. Continue with platform-independent APIs and configurable capacities, refining concrete limits with later use. These missing fixed values are therefore not a reason to stop the authorized algorithm/storage work or request the same approval again. Do not invent hard-real-time guarantees.

Reusable update workspace, 2026-10-12 02:30 +02:00. User says continue; all catalogue implementations remain authorized. No fixed deadline/platform is required: use configurable capacities. Core fceee42 is clean, one local commit ahead of freshly fetched feature upstream and two ahead of main. Reuse the checkout; no new commit/push request is inferred for this continuation.

Add public ValueUpdateWorkspace in the existing value-update owner and export it from execution. Its fallible with-capacity constructor reserves target/order, changed-ID, staged-value, symbol-occurrence and removed/added-hash buffers. The existing with-value-updates creates a temporary growable workspace and delegates to the same implementation as the new workspace-taking method. Preserve existing 4096-edit/65536-value-node/32-depth/1-MiB-text caps. Bound configured update capacity by the existing edit cap; reject reservation overflow/failure with LimitExceeded. Symbol capacity counts old plus new string occurrences, not distinct strings.

Sort (DeclId,input-position) pairs and find the earliest duplicate position, then perform validation in input order with the same target/duplicate/value precedence. O(k log k), no hash-collision assumption or domain-sized bitmap. The private IR leaf setter borrows/drains its staging Vec

only after all targets pass; returned records own their values. Clear workspace contents on every ordinary Result path; retain only buffer capacity, never old model references or payloads.

Reuse SymbolDelta storage in its resolution owner. Give the single macro-generated read/write symbol schema typed visitor error propagation, so occurrence exhaustion is distinct internally from an invalid symbol and maps to existing LimitExceeded. Short-circuit scalar string visits without an auxiliary stack. Nested list walks may still allocate; do not call the complete update allocation-free. Reuse the semantic-delta's removed/added hash vectors in semantic_digest.rs; output arrays and the canonical full digest fold remain unchanged. No global allocator or unsafe.

Discovery: current SymbolDelta already owns delta collection; extend it. No existing value-update workspace or semantic-delta buffer owner was found. Tensor resource arenas own backend/numeric buffers, not this model-bound scratch. Touched: CORE.YAML, this plan bundle, generated SYMBOLS.md, README.md API example, `hymeko_core/src/module_store/execution.rs`, `hymeko_core/src/module_store/execution/value_updates.rs`, `hymeko_core/src/resolution/symbols.rs`, `hymeko_core/src/ir/semantic_digest.rs`, `hymeko_core/src/ir/ir.rs`. No consumer signature migration: additive API, same existing call behavior.

Test constructor zero/overflow bounds; exact capacity and one-over; old/new repeated symbols; cleanup after early/late error; model switching and retained readers; whole-output/digest parity; duplicate-error precedence versus a small HashSet oracle; stable scratch allocations across reuse; private setter atomicity. Run targeted tests before Core library/IR/doctests/Clippy and portable gates. No timing campaign. PDF/report archive, 16-GiB resource ceiling. Worst inputs include one huge old nested value and many repetitions of one symbol: capacity is occurrence-based and does not pretend to cap all model allocations. ETA about 30–40 minutes: implementation 15, correctness 10, API example/report 5–15. Roll back only these named hunks, preserving the committed preceding algorithms.

Workspace verification closure, 2026-10-12 02:53 +02:00. Implemented the additive ValueUpdateWorkspace/with-value-updates-in facade and one shared update implementation. Six configured buffers retain their storage through success, capacity rejection and model errors; no payload/model references remain afterward. Private leaf application drains the borrowed staging Vec only after validating all bindings. Symbol and semantic deltas reuse their owning buffers. Reader/writer visitors retain one field schema with typed errors; boolean live marks and exact counts share one checked SymbolMarks implementation.

Final correctness: 198 library, 181 unified-IR and 8 doctests passed (9 ignored). All 1,555 input sequences through length four preserve the original HashSet validation order. Exact old-plus-new repeated-symbol capacity, zero and overflow constructors, early/late errors, model switching, retained snapshots and six storage identities are covered. Clippy with warnings denied, formatting and symbols pass. Duplication is unchanged from HEAD: 448 clones / 8,529 lines, below stored 484 / 10,411. A transient three-clone increase was consolidated before these final checks. No benchmark/profiler or downstream migration run.

Preflight candidates were addressed in their owners: ValueUpdate's existing module now owns reusable scratch; the incidence validator/ValueR are not scratch owners. Existing SymbolDelta was extended; no equivalent semantic-delta owner existed. Core manifest (191 files): `e238f3f79dc5b52e19d13fa3f41d5c29c7e16d9ab82e2603e2c397650a79f633`. PDF: `reports/2026-10-12-upd.pdf`; evidence and source snapshots in the matching directory, archived at close. New public configuration bounds the six scratch buffers, not snapshot/validator/nested-payload allocation or wall-clock time. No new commit or push in this continuation.

Evening algorithmic completion, 2026-10-12 04:03 +02:00. User explicitly requests carrying the algorithmic tasks through. All prior authorizations remain valid; capacities are configurable and there is no fixed platform/deadline gate. Core fceee42 remains one ahead of freshly fetched feature upstream; preserve the pending workspace changes. Sequence: model-bound

validation reuse; inverted symbol-use maintenance/selective remapping; bounded snapshot retirement; Graph visitation without generation wrap; budgeted reclamation in Tensor's existing pool. Paged IR and TLSF are conditional alternatives: examine the contiguous slice contract and allocator/backend boundary before selecting them, and record an explicit disposition rather than leave an unexplained TODO. No new Git publication is inferred. Work is sequential in existing checkouts.

First phase: the hierarchy/index validator reads provenance, topology, dispatch and resolution paths; model validation reads facets/ports and operator identity, not leaf values. Add a private opaque proof and lazy cache to CompiledModel. For value-only updates without symbol-ID remapping, validate the immutable base once and carry the proof into the derived model. Fresh compile/decode starts unproven; decoder limits differ, so do not assume its admission proves the update validator's default limits. Dense ID remapping initially requires full candidate validation again. Errors remain InvalidModel; failed bases may cache that result. The proof stores no parent Arc, avoiding historical-model retention.

Touched in this phase: CORE.YAML, this plan bundle, generated SYMBOLS.md if needed, `module_store/execution.rs` and `execution/value_updates.rs`. Test cold/branched/repeated edits, remapping, invalid hierarchy and model facets, and source/snapshot independence. A test-only counter observes actual full validation calls; full validators remain independent result oracles. Existing workspace error ordering must stay unchanged. Run targeted Core checks before moving to later owners. Initial phase ETA 15–20 minutes; the full multi-owner sequence requires several implementation/test cycles. No hard wall-clock or world-ranking claim. Existing resource cap 16 GiB. Final PDF and raw evidence archived; rollback only this continuation's hunks, preserving prior work.

Selective symbol maintenance, 2026-10-12 04:15 +02:00. Validation reuse passes its branch/remap/error/parent-retention tests. Next, extend SymbolUsage with a flat sorted inverse index of (symbol, record group, occurrence count). Build it once through the same typed field schema, using a BTreeMap to aggregate repeated index-prefix occurrences without an unbounded raw-occurrence staging vector. Retain no strings or earlier models in the index. Separate declaration names/annotations, roots, base references, arc annotations/ references, facets, resolution index and imports so dependencies are explicit.

Value deltas carry their declaration-annotation record key. Sort/combine them and merge into the immutable inverse rows; unchanged per-record multisets share rows. This merge is O(U+R log R), not a false O(R) claim, where U is unique symbol-record uses. Counts and rows may still copy linearly. Dense remapping selects only record groups containing moved IDs, reuses the existing typed rewrite helpers, and detaches metadata only if its IDs change. Classify path/validation dependencies separately: annotation-only remapping need not rebuild canonical path layout or repeat structure validation. Update the inverse row IDs in the same monotone mapping order. Keep full compaction as an independent oracle and CBOR admission path; there is one field schema, not two inventories.

Touched additionally: `resolution/symbols.rs`, `ir/semantic_digest.rs` argument semantics and workspace's test-only storage observer. Tests cover per-record count equality against full rebuilding, swaps whose global counts are unchanged, static metadata symbol coverage, remaps of dynamic tail strings without topology copies, and remaps of names/prefixes that must invalidate proofs/layout. Benchmark interpretation must account for inverse index memory and first-use cost. No new public signature or wire change.

Bounded snapshot retirement, 2026-10-12 04:43 +02:00. The first Core phases pass targeted symbol/update oracles, 203 library tests and strict Clippy. Add a caller-owned SnapshotRetirement queue with fallible configured capacity and a bounded number of root inspections per reclaim call. Existing commit keeps its signature/consuming-error behavior. New commit-retaining returns the candidate on full/rejected publication, so callers choose retry or destruction

on their worker. There is one shared publication implementation. Preallocate the next Arc before the write lock; retain foreign/stale/poison/revision error ordering for the existing entry point. Invoke replacement handling after releasing the lock.

Reclamation rotates a preallocated VecDeque and uses Arc::try_unwrap, not a racy strong-count test, so concurrent weak upgrades cannot move the final snapshot-root destruction to a reader. A live reader keeps the root queued. A full queue refuses publication; no unbounded growth or implicit worker/thread is introduced. Capacity counts roots, not bytes; dropping a whole model can still be O(model size), so the reclaim call belongs on a suitable caller-owned worker. Copies of IR/metadata held independently by consumers retain their own lifetimes. Dropping the queue is an explicit teardown, outside the deferred-reclamation contract.

Touched: execution.rs, existing execution_tests.rs, public API example in README, this bundle/SYMBOLS.md. No equivalent retirement owner found in Core; numerical arena epochs are Tensor's distinct resource lifetime. Test zero/full capacity, candidate recovery, retained readers, weak references, bounded rotation, stale/ foreign/poison/overflow errors and post-lock callback execution. No new global allocator or public PublicationError variant. ETA 15–20 minutes for this phase.

Opt-in paged declarations, 2026-10-12 05:02 +02:00. Keep the existing IrTable as the sole record owner. Add an opt-in paged storage variant with a shared directory of shared fixed-size pages. Default publication stays contiguous; add CompiledModel::with-declaration-pages with a checked power-of-two page size. IDs, iteration order, snapshots, wire and identities remain unchanged. Clone shares the directory; indexed mutation copies directory handles and only affected pages. Generic mutable Vec access deliberately makes the table contiguous; read-only Vec/slice access lazily materializes a shared contiguous cache. The cache is invalidated on mutation and never becomes a second semantic authority. A clone function installed only for Clone records permits preserving generic read/Deref interfaces without unsafe code.

Use a streaming exact-size/double-ended iterator and indexed record access to avoid flattening in Core's normal paths. Keep Vec range operations through explicit range indexing. SkeletonForest borrows IrTable rather than forcing a slice. Existing mutable construction remains owned/flat. SemanticDigestState is immutable and may be Arc-shared when only physical storage changes; no parent program is retained. Explicit contiguous access may cost O(N) time/memory, so do not advertise paging as universally faster or change the default blindly.

Touched additionally: ir/table.rs, ir/ir.rs accessor, ir/canonical_hash.rs, execution.rs and `value_updates.rs` cache construction; existing table/update tests, README and benchmark harness only if a layout control is added. Tests cover page boundaries, directory/page sharing, cached-slice invalidation, range mutation, streaming/reversed/consuming iteration, unique movement, serde parity, invalid page sizes, complete update/CBOR parity and retained readers. Array bound checks and all existing validators remain in force. Preserve the current non-paged candidate before evaluating layout read/update tradeoffs.

Pool reclamation continuation, 2026-10-12 05:20 +02:00

Existing owner: Tensor compute arena Pool, already generic across host and device. Extend it with a rotating VecDeque retirement queue and a configurable maximum number of epoch checks per call. Existing full reclaim delegates to the same implementation. Incomplete epochs rotate without starvation; only completed resources enter the existing free-list policy. Budget covers checks, not free-list eviction, allocator, backend destruction or a wall-time deadline. Do not silently claim those bounded. Zero budget is a no-op; initial length caps each pass. Tests use existing deterministic Device/Rig fixtures: fairness behind incomplete heads, bounded checks, pending/session safety, eviction/counters and full reclaim. No dependencies or global allocator. TLSF is not installed: ArenaBackend resources are opaque objects, without an owned raw

arena, split/coalesce/alignment contract. A framework must not silently select the application's process allocator. Existing size classes/pool and bounded update workspace address current concrete users. Tensor changes: CORE.YAML, existing pool.rs, existing compute_arena.rs tests, README and symbol index if applicable. Fresh fetch shows clean main 774ed7d; no branch publication. ETA 15 minutes, 16 GiB cap. Core gates first, then pool tests, strict crate Clippy and local Core/WGPU consumer check using one Tensor identity. Rollback only named hunks. Combined PDF and archived evidence close the task.

Validation closure repair, 2026-10-12 05:34 +02:00

Strict cognitive-complexity checking exposed the existing coherence validator at 28/25. Inspection also found that a short dispatch table emits a length finding and then indexes past its end. Add a reproducer before changing the implementation. Extract record-binding validation into a private helper in the same owner, visit only present slots after reporting the length, and preserve finding order on all previous non-panicking inputs. No schema or dependency change. Existing exhaustive coherence oracle plus the new malformed-table test, full Core tests and strict Clippy cover the repair. This extends the authorized validation owner, not a second validator. ETA 5 minutes. The benchmarked binary remains explicitly identified as preceding this cold validation repair; no retiming claim.

Test-only gate closure, 2026-10-12 05:38 +02:00. The production complexity gate now passes; three unchanged test functions exceed 25 because of accumulated assertion groups. Extract their symbol, declaration accessor and per-node incidence assertions into named helpers in the same files, without removing an assertion or changing a fixture. Affected existing tests: ir_transformation_tests.rs, validation/corpus.rs, prepared_ir_backbone.rs. Re-run the whole IR test target and the all-target strict complexity gate. Graph formatting removes only two pre-existing blank lines in octree.rs. No production behavior or benchmark binary changes from these test/style edits.

Digest storage continuation, 2026-10-16 23:13 +02:00

Scope and ETA. Continue the user-authorized Core optimization in the existing semantic digest owner. ETA 15–20 minutes for implementation, correctness, static gates and a PDF report. Current source copies both full declaration-hash arrays even for an unchanged value. Reuse IrTable with 256-entry copy-on-write pages for declaration-ID lookup, and Arc sharing for the sorted fold input when unchanged. Only changed hash pages detach; retain the existing sorted delta merge and encoder. The final fold and sorted merge remain linear. No latency improvement is asserted without a new measurement; this continuation runs correctness checks only.

Authority and checkout. User requested continuation of optimizations; existing incremental-semantic-digest and algorithmic-completion approvals apply. Both remotes fetched successfully. HEAD fceee42 is 2 ahead/6 behind origin/main, 1 ahead of its upstream. Incoming main commits concern generators; the dirty algorithmic checkout is preserved without merge, reset, branch switch or push.

Files and contracts. Core semantic_digest.rs and existing update tests if needed, this four-format plan, and generated evidence under the existing algorithmic-completion report directory. Protected owner: hymeko_core, under the existing and current user approvals. No public API, dependency, wire, version, validation, input limit or canonical hash change. No other repository is modified.

Tests and risks. Add deterministic pointer-sharing regressions before implementation, then compare incremental results against Canon::hash at empty, partial and multiple page boundaries, signed zero, unchanged edits and retained branches. Reuse IrConstruction for fixtures. Run targeted digest and checked-update unit tests, semantic/CBOR integration filters, strict all-target Clippy, formatting, symbol and clone gates. Prior gate: 455 clones / 8597 lines, baseline 484 / 10411. For N declarations and k dirty pages, hash payload copying becomes at most 256k hashes plus ceil(N/256) directory handles; sorted storage and hash fold stay O(N). Small inputs pay page metadata; no unmeasured choice is called optimal. Worst case 4096 edits spread across pages can copy most hashes. Preserve workspace capacities and symbol remapping. Max process RSS remains 16 GiB; no benchmark or profile is started. Source backups and the task-only patch provide rollback; preserve all pre-existing dirty work. Report PDF and evidence are archived and verified.

Discovery. Existing SemanticDigestState, IrTable, IrConstruction, value-update tests and algorithmic reports were searched and read. Extend these owners; no competing storage abstraction, runtime helper module or standalone benchmark runner is introduced.

Digest storage measurement, 2026-10-16 23:45 +02:00

Authorization and ETA. User explicitly requested measurement of the completed optimization. ETA 15–20 minutes: two matched builds, smoke checks, approximately 10 minutes sequential Criterion runs, conditional short profiles, analysis and PDF/archive. Reuse interning_bench, measure_peak_memory.ps1 and criterion_report.ps1. Production code is fixed. Extend the benchmark with an unchanged numerical edit and initial retained digest construction. A cold paged CompiledModel template is created before any semantic cache access; each untimed setup clones it through with_declaration_pages, keeping its private cache empty. The timed initial-state operation is semantic_digest; destruction is outside timing. This characterizes initial state on paged declaration storage, not source compilation.

Matched versions. Original production source is the verified October 16 source snapshot, matching the optimization's before hash. Candidate differs only in semantic_digest.rs. Both use the identical extended benchmark, manifests, lockfile, source path, release bench debug=1 profile and dependency artifacts. Build sequentially in an isolated generated source directory using Core's existing target directory, then retain both EXE/PDB pairs. Do not swap the user's checkout source. Fresh fetch of origin/github succeeded, same fceee42 branch divergence; preserve dirty work.

Matrix. Sizes 1000 and 50000; numeric, text, unchanged numeric edits, each with flat/paged declarations, plus initial state: 14 cases. Five independent process runs per version and size; alternate AB/BA and size order. Twenty warmed Criterion samples per case, 1 s warm-up and 3 s measurement. Dedicated –bench opt-in; smoke both variants at 50k with –test before timed runs. Rayon one thread, logical CPU 2/mask 4. Observe CPU load/power state; other user applications remain active. 16 GiB userspace high-water guard, 100 ms polls, 10 s progress, 150 s process timeout. Expected peak below 1 GiB. No concurrent benchmark, build or profiler.

Acceptance and follow-up. All exit codes, fixture source hashes/counts, full/incremental digest assertions and resource caps must pass. Report per-process medians, IQR and worst batch mean plus median of five process medians; do not treat inner samples as independent runs. More than 10 percent repeat spread or regression requires examining affected cells, not blindly expanding the grid. Attempt bounded Samply profiles of representative before/after cases for improvements above 10 percent; profiling is separate from latency. If ETW cannot

record, disclose the lack of profiling attribution rather than manufacture a flamegraph. Keep raw records, commands, hashes and build provenance. No world-ranking or hard-real-time claim.

Files and rollback. Existing interning_bench.rs, four-format plan and existing digest-sharing report/evidence. No production/dependency/API changes. Current implementation report is preserved before replacing it with an expanded PDF. Rollback only this task's named harness/plan hunks and generated evidence. Existing quality checks and canonical fixture assertions exercise the new measurement paths; no unrelated test or consumer campaign. Archive final files using the shared provider.

Virtual semantic fold, 2026-10-17 02:37 +02:00

Scope and ETA. User requested implementation of the virtual-hash idea and its possible gains. ETA 25–35 minutes for design, implementation, structural correctness gates and PDF. Extend SemanticDigestState with a deferred canonical byte stream and persistent BLAKE3 subtree cache. No new performance run: current request authorizes implementation; operation-count and pointer-sharing assertions are correctness tests, not timing claims. Current public Copy CanonicalIdentity requires materialization before a checked update returns. Preserve this boundary; private fold states may defer and combine invalidation, but do not claim that the public update transaction becomes deferred. No new public hash type or encoding.

Byte contract. One virtual-input owner emits the existing model header, four domain/length headers and sorted 32-byte contributions. Existing full hash and partial range reads use this encoder. Freeze known answers from the unchanged implementation first. Use pinned blake3 1.8.7 hazmat helpers, correct byte offsets, canonical tree splits and root/non-root distinction. Cache leaves cover at most 16 KiB aligned BLAKE3 subtrees, including partial final leaves; no unsafe casts or full concatenated byte-buffer. Compare with standard blake3::hash and old vectors.

State and invalidation. Track the first/last changed sorted declaration positions during the existing delta merge. Convert them to exact byte offsets; unchanged prefixes, suffixes and concrete-record sections share hash subtrees. Pending ranges union across unevaluated states, sharing the last materialized tree without retaining a chain of parent models. Lazy evaluation uses OnceLock. A final state may retain its seed tree and its current tree, whose unchanged nodes share. Cold state builds once; repeated reads reuse it. Small inputs use direct BLAKE3.

Files and authority. Existing semantic_digest.rs and value-update docs/ tests as needed; new private semantic_digest/fold.rs (separate cache concern), this plan bundle, CORE.YAML approval record if needed, generated PDF/evidence in reports/2026-10-12-algorithmic-completion/virtual-hash. User's explicit request and existing incremental-semantic-digest/algorithmic-completion approvals cover Core. Fresh origin/github fetch succeeded; fceee42 branch remains 2 ahead/6 behind main, 1 ahead upstream. Incoming changes are generators. Preserve all pending work; no merge, checkout replacement, commit, push or consumer/dependency modification.

Verification and risks. Deterministic boundary/parity tests cover short, exact/partial blocks, power-of-two tree edges, domain boundaries, raw-ID mode, local and broad reorderings, cold chains, cancellation, retained readers and concurrent first access. Assert bytes read for dirty ranges and unchanged subtree sharing. Original digest/update/CBOR/corpus gates remain authoritative. Strict Clippy/format/symbol/duplication checks; source-only rollback from backups. The final sorted merge remains O(N); rank changes can dirty a long interval, so no universal logarithmic update bound. Cache increases initial work and memory; 16 KiB is an unmeasured

storage granularity, not an optimum. Max process RSS stays 16 GiB, bounded test fixtures and two Cargo jobs. No benchmark/profile is launched. Archive report and evidence through the existing provider with content verification.

Discovery. No semantic subtree cache exists. Reuse Canon and its semantic owner; structural CanonHashCfg encodes a different hash contract and is not a replacement for this fold. Existing IrTable remains the declaration store.

Night continuation: unchanged values, 2026-10-17 03:16 +02:00

Scope and ETA. User requested continued overnight Core optimization. ETA 15–25 minutes for this bounded implementation step, tests and PDF. The prior no-op profile shows remaining Arc/page and record-copy work; the IR setter still writes unchanged values. Reuse the existing batch setter and workspace. Stage a private Keep/Replace leaf-value edit, validate every target before commit, and write only replacements. Compare resolved values bit-for-bit, including signed zero, using iterative nested-list comparison. Clone payloads only for replacements.

Contracts. The entire input validation pass remains first: duplicate and invalid-value precedence, size/depth/text caps unchanged. Keep targets still undergo the setter's binding/leaf preflight. SymbolUsage processes every requested target, including unchanged strings, to preserve configured occurrence budgets and unreachable-symbol cleanup. Afterward the existing changed-ID scratch is repopulated with actual replacements for incremental hashing. Reuse the target buffer after duplicate checking; bounded capacities do not grow. Source snapshots and all canonical/CBOR bytes remain valid. Publications still advance revisions. No general short-circuit around validation or symbol compaction is introduced.

Owners and files. Core ir/ir.rs (private edit intent, value comparison, existing setter and tests); execution/value_updates.rs (staging, actual dirty IDs, contracts and regression tests); semantic_digest.rs test callers adapt to the private setter type. Existing four-format plan and generated reports in reports/2026-10-12-algorithmic-completion/unchanged-values. No public API, dependency, wire, hash version or other repository change. Existing Core approvals and current continuation authorize the edit. Fresh origin/github fetch succeeded; fceee42 is 2 ahead/6 behind main and 1 ahead of upstream. Preserve prior dirty work.

Verification. Add a public-path pointer-sharing regression before the implementation; it must fail on current eager writes. Then cover flat/paged no-ops, mixed batches, nested lists, strings, clears, signed zero, nonfinite rejection, unchanged-string workspace exhaustion, missing record bindings, symbol cleanup, old readers and publication revision. Match full hashes/CBOR and actual rehash counts. Existing IR/digest/update/CBOR/frozen contract tests; strict Clippy, format, symbol and duplication gates. No benchmark/profile: current request continues implementation; no new measurement is explicitly requested. Report structural copy avoidance, not a latency percentage. The prior measured code remains archived.

Bounds and risks. Iterative comparison uses O(depth) frames, with no frame allocation for scalar or flat-list roots. Public update limits remain 4096 edits, 65536 new value nodes, depth 32 and 1 MiB text. Input data can be unchanged and still invalid; validation cannot be skipped. Dirty mixed flat tables can still copy untouched records on the same table, and symbol compaction can still detach storage. Returned models/metadata may allocate; no allocation-free or RT claim. Use at most two Cargo jobs and small deterministic fixtures within the 16 GiB process budget. Roll back only task hunks using preserved sources. PDF and raw verification artifacts are archived with the shared provider and byte verification.

Discovery. Existing ValueR, Ir::set_leaf_values, ValueUpdateWorkspace, SymbolUsage and semantic delta are the owners. No existing bitwise comparator or private preserve/replace edit intent was found. Extend these; do not add another model, unchecked setter, global allocator, or versioned identity type.

Unified construction and compact references

Created-at: 2026-10-17 19:45 +02:00. ETA 90–150 minutes: shared construction and capacity tests, compact reference/serialization migration, Core correctness gates, then affected Graph/Transformation consumers and PDF/archive. User explicitly requested lazy capacity/allocation, byte-sized sign, annotation IDs and vector weights, and challenged construction-path integration. No new benchmark or profile is requested in this implementation step; compile-time layout and allocation-state assertions are correctness checks, not latency claims.

Audit. One Core checkout, fceee42 on perf/checked-value-update-symbols. All 195 measured source files still match the previous comparison manifest. The comparison used current source but a manual benchmark builder, bypassing IrConstruction and lowering's existing preallocation. Core has one production lowering owner and two manual benchmark builders; Emitter adds a manual editor bridge. These are not four independent compiler implementations. Snapshots and checked value updates are separate legitimate operations. Keep invalid fixtures and independent oracles explicitly test-only.

Capacity and loading interpretation. Known graph dimensions are a capacity plan, not initialized placeholder entities. Extend IrTable to retain unallocated capacity hints until mutable population needs storage. Read-only use must not allocate or detach. Empty optional stores stay empty; sharing/snapshots, serialization and counts ignore unused capacity. Extend IrConstruction as the single mutation owner for root/child nodes, edges and arcs; migrate the two benchmark builders and editor bridge instead of introducing another builder. Existing indexed lowering preserves stable named declaration IDs and forward refs. No file-backed out-of-core loader or virtual-memory pager is claimed.

Reference layout. Replace embedded SignedRefR payloads with one compact canonical reference: target, typed IR-owned annotation/weight-vector IDs, explicit one-byte algebraic sign and one-byte port direction. Reuse existing typed ID machinery where possible; target native record budget at most 32 bytes. Empty annotations and absent weights require no payload allocation. Present weights always resolve to a vector, including a present empty vector. Preserve None versus Some(empty), signed zero, ordering and multiplicity. Keep direction independent of sign: In/Out/Unspecified is used by executable-model validation and diagrams. Construction drafts may own incoming payloads; they are transient inputs, not a second retained graph. Annotations and weight vectors have one canonical IR owner.

Compatibility. Preserve existing CBOR/JSON field order and enum wire shape through borrowed owner-aware views; do not serialize internal payload IDs. Deserialize existing payloads into the compact store. Preserve canonical semantic and structural digest bytes and versions. Update the hash, symbol visitor, incidence views, validators, tensor extraction and writers to resolve payload IDs through Ir. Shared payloads must be remapped exactly once during symbol compaction; only live payloads keep symbols alive. Snapshot copies preserve independent readers. No global mutable registry, unsafe packed references, versioned aliases or new runtime dependency. Existing raw invalid-fixture behavior remains test-owned.

Owners and files. Core common/ids.rs; ir/ir.rs, table.rs, lower.rs, new reference.rs and its focused wire/test helpers if needed; resolution/resolve.rs and symbols.rs; canonical_hash.rs, semantic_digest.rs, incidence.rs, coherence.rs; tensor/tensor_val.rs; util/pretty_print.rs; cbor boundary/validators; affected tests, benches and examples. Extend existing canonical owners. Then migrate Graph and Transformation reference readers/construction clients. Protect unrelated dirty work, especially geometry/simulation and other agents' work; no RL edits. Update approval/discovery/symbol records and this existing four-format plan.

Verification. First capture the old record-size regression and byte oracles. Test deferred capacity without live-element growth, empty stores, first population, clone/write isolation, construction failure atomicity and hierarchy. Test reference sign/direction size, default/annotated/vector cases, vector presence, foreign/out-of-range payload rejection at checked boundaries, shared-payload remapping, wire bytes, digest known answers, symbol lifetime and old snapshots. Run selected non-measurement Core unit/integration/corpus/codec gates and strict Clippy/fmt/symbol/duplication. Then verify relevant Graph/Transformation consumers against this exact local Core through recorded temporary CLI overrides; never present an old Git pin as having these uncommitted changes. No commit/push/merge.

Git and resources. Fresh fetch succeeded for Core, Graph and Transformation. Core is 2 ahead/6 behind origin/main; Graph is 1 ahead/4 behind origin/main; Transformation is 2 ahead/0 behind main. Dirty checkouts stay in place. Keep two Cargo jobs, bounded fixtures and the 16 GiB process memory limit. No speculative speed claim. Preserve before-sources and task patches for rollback; archive PDF and correctness evidence through ReportArchive with byte verification.

Builder ownership correction, 19:47 +02:00. User reiterates that construction entrypoints belong to owner structs/traits with build() methods. Retire free-function builder bodies after migrating all callers. Input adapters may differ, but table mutation and capacity policy stay in IrConstruction/IrTable. The measured legacy benchmark code remains a frozen report artifact, not a production fallback. Add caller/ownership checks so a normal benchmark or emitter cannot silently restore its own table/binding algorithm.

Deferred loading completion, 21:04 +02:00. Expose a lazy holder for any existing IrBuilder input: retain the known capacity plan without populating an IR; first load delegates to the same build method and caches either its output or its typed error. Repeated reads never rebuild. Dropping an unloaded holder performs no construction. This is demand construction of the whole input, not per-page disk I/O. Verify successful and failing population counts and unchanged canonical output. No additional construction algorithm or dependency is added.

Matched native comparison and default-path audit

Created-at: 2026-10-18 02:06 +02:00. ETA 60–90 minutes. User requests fresh numbers and the better implementation as the default. Preserve semantic defaults: undirected/unit PageRank remains selected unless the caller requests directed routing. No precision relaxation, legacy IR builder, new production backend, profiler, commit, push, RL edits, or other-agent work is included.

Construction. Build the current Core benchmark using the sole SyntheticIrBuilder/IrConstruction path and compact 32-byte references. Compare against the unchanged archived hypergraphz native adapter at pinned commit e45b7b80e3ce302937a56760d9fe76016ef08e26 on 1,000 disjoint hyperedges of 1,000 vertices each. Verify every member and normalized checksum before timing. Five fresh processes per backend, build and build/drop, CPU affinity, raw Criterion samples, peak memory and source/binary hashes. Historical figures are identified as historical and are not substituted for fresh same-host results.

Active operations. Extend the existing Graph evidence client and process runner with a benchmark-only native ABI calling unchanged upstream computePageRank and getConnectedComponents. Both use identical canonical fixture membership exported outside timing. Uniform personalization, unit weights, damping 0.85, global L1 error bound 1e-10, cap 200; upstream iterate-difference threshold is converted by (1-alpha)/alpha. Distinct members per edge avoid the upstream set-incidence versus HyMeKo multiplicity difference. Compare full rank vectors and normalized whole-component partitions outside timing. Compare one-shot PageRank including preparation/workspace/result cleanup on both sides; show HyMeKo reusable-workspace solve separately with its boundary explicit. Reverse indexes are built before component queries on both sides. Upstream BFS follows ordered pairs and is not equated with HyMeKo hyperedge reachability.

Measurements and gates. Use existing sampler (two warmups, nine samples), five fresh processes per cell, one CPU, 16 GiB observed process guard, 180-second process timeout, and source drift checks. Start with a full-scale smoke. Fixtures: 65,536 vertices, one million memberships, arities 2/32/1000. No uniform regular-disjoint PageRank speed claim: its uniform start is already stationary. Small native parity tests cover isolates, singleton/empty edges, asymmetric degrees, and disconnected components. Existing directed correctness stays active; directed external comparison is inapplicable because upstream PageRank is unsigned. Run scoped correctness, Clippy, formatting, Python controller tests/types/lint, symbol and duplication gates. No production optimization without evidence.

Delivery. Audit normal callers and default configuration, preserving one production construction and PageRank implementation. Record fresh Git fetches, dirty source identity, temporary local Core Cargo patch and restored lockfile. Produce PDF, raw numerical/source/binary evidence and byte-verified shared archive. Reuse the current checkouts and shared reporting tools. A slower result is reported as such; selecting different mathematics is not an optimization.

Completed comparison, 2026-10-18 02:31 +02:00. Completed 75 Graph measurement processes (675 samples), five smoke cells, and 15 construction processes (600 samples), with full result/source checks. Current construction 80.03 ms versus archived builder 79.83 ms; build/drop 85.09 versus 93.14 ms. Native hypergraphz construction 61.07 ms. Components favor HyMeKo (2.17/6.06/89.19 times); PageRank favors the pinned upstream (7.63/4.57/3.13 times), at identical iteration counts and precision. The small singleton-edge component case exposes an upstream short-window indexing limitation; the adapter explicitly rejects that operation, without modifying the upstream algorithm. All timed cases are within the common domain. Production sources

are unchanged, one canonical owner remains the local default. Graph's published Core pin is still old; this run records a temporary local patch, not a completed publication. No profiler, production optimization, commit or push.

Shared virtual incidence preparation and PageRank integration

Created-at: 2026-10-18 17:29 +02:00. ETA 45–75 minutes: shared projection and operator integration, directed/numerical/storage regression gates, PDF and archive. Authority: the user explicitly requests applying the shared virtual sparse operator approach to PageRank, building on the existing Core owner. Earlier compact-reference approval covers correcting the demonstrated retained-capacity error. No benchmark, profiler, commit, push, RL or other consumer migration is included. This is an ownership/integration change with structural correctness checks; no speed factor will be claimed without a later authorized comparison.

Discovery. IncidenceIndexData owns reverse CSR coordinates; PreparedIr/IncidenceView bind it to canonical Ir. IncidenceOperatorPlan already prepares signed transpose coefficients. PageRank currently owns repeated target resolution and per-incidence role dispatch. Extend this family; pairwise Csr and uniform-arity f32 SignedHyperIncidence have different contracts. The new private incidence/projection.rs submodule keeps the existing large owner readable; its public types are re-exported by ir::incidence. No new crate or dependency.

Shared representation. Add IncidenceProjection with immutable, reference-counted arc offsets and native-size numeric target indices, paired with its Ir. Explicit domains are canonical declarations or canonical nodes. Native usize indices preserve existing size limits; no unchecked u32 narrowing. Validate the selected target domain once, preserving order and duplicate slots. The existing reverse index lazily caches each projection domain, sharing only structural data across topology-preserving model updates. Signed coefficients stay attached to the operator/Ir, never in the shared structural cache. A direct projection build avoids forcing a reverse index on an algorithm that needs none.

Shared execution. IncidenceRow owns gather/scatter primitives with a small accumulator trait: ordinary f64 for the existing signed operator, the existing compensated sum for PageRank. Existing IncidenceOperatorPlan gains lazy forward coefficients derived from its checked transpose coefficients and uses these shared rows for Bx. Its transpose implementation stays intact. PageRank uses the same row operations, fusing each arc's gather and scatter to avoid an extra arc-sized temporary. No B-transpose-B or clique is materialized.

PageRank. The ordinary build method creates the Core node projection; a constructor from an existing projection permits reuse by multiple operators. Remove per-iteration DeclId decoding and sign/port matching. Prepare unit, directed-unit role bits, or normalized component rates once. Dispatch outside the incidence loop. Preserve self-return, multiplicity, independent sign/port semantics, selected vector component, dangling handling, personalization, compensated arithmetic and convergence bounds. Preserve original fold and multiply order; no precision relaxation or retained old/new kernel switch.

Storage correction. store_references currently reuses the 96-byte ReferenceDraft allocation for 32-byte records: 1000 records retain capacity 3000. Compact the completed vector at the canonical owner boundary; allocation may be resized or moved once. Test the retained IR vector directly (not a clone), payload presence, wire/hash semantics and empty input. Existing IR record identity is unchanged. Do not add a per-consumer shrink workaround.

Files and tests. Core: CORE.YAML, ir/incidence.rs, its projection submodule/tests, ir/reference.rs and reference/tests.rs, existing prepared-IR and precompute correctness tests, README/SYMBOLS and this plan. Graph: CORE.YAML, direct_ir/pagerank.rs and module documentation, existing pagerank.rs and pagerank_alloc.rs tests, README/SYMBOLS and this plan. Keep frozen

earlier measurement artifacts. Add Core domain/cache/adjoint/duplicate/empty/invalid projection tests and shared prepared-forward parity. Graph independent dense oracles cover all route/weight modes, isolated/interleaved IDs, extreme weights, nonconvergence and guards; add shared-projection parity and actual default-path allocation checks. Warm applies and rank solves remain allocation-free.

Verification and resources. Inspect exact test targets; run only non-measurement correctness tests, compilation, Clippy, fmt, symbols and duplicate gates. Two Cargo jobs, bounded deterministic fixtures and the 16 GiB process cap. Graph uses a recorded temporary local Core patch and restores Cargo.lock bytes; the older published dependency pin is not misrepresented as updated. Extra projection storage is O(I+E); cache population and failures are explicit. No retained payload copies, global registry, unsafe indexing, dependency cycle or new IR builder. Wrong-domain or stale pairing must not silently succeed.

Git and rollback. Fresh origin/github fetch succeeded in both repos; Core fceee42 is 2/6 and Graph 921d1d9 is 1/4 versus main. Each has one worktree. Preserve all earlier dirty work. Before-files and task-only patches live under the existing comparison report family's shared-incidence phase. Report PDF, source/binary/test receipts and byte-verified ReportArchive copy close the task.

Exact-count preparation and seeded ordered reductions

Created-at: 2026-10-18 19:13 +0200. ETA 35–55 minutes: implement two bounded changes and targeted selection in the existing runner, numerical/allocation gates, release comparison, PDF and verified archive. User authority: continue the competitive PageRank optimization on the shared Core IR after the completed comparison.

Evidence and scope. The last exact-source campaign has full-call medians 206.23/46.07/34.63 ms versus hypergraphz 170.67/45.11/41.10 ms. Preparation still walks IR references for unit destination sums and uses a compensated outgoing-count buffer. Every gather starts compensated summation at zero. Remove this proven redundant work; do not claim a profiled percentage.

Core contract. Add an additive IncidenceAccumulator::from_first method whose default is Default + add. Ordered sum initializes from its first term and keeps all later terms and callback ordering unchanged; empty input still returns Default::total. PageRank's private compensated policy supplies an algebraically equivalent first-term state, retaining signed-zero and nonfinite behavior. No new kernel, graph, unsafe indexing, dependency or precision mode.

Unit preparation. For undirected unit weights and at most 2^53 incidences, obtain destinations from row lengths and count outgoing membership with the existing shared f64 scatter. Each integer addition is exact in this bound. Skip the redundant reference scan and compensated temporary vector. Larger counts and other policies use the existing general preparation, preserving its domain and error checks; no artificial graph-size limit. Directed roles, selected weight components, duplicates, empty arcs and isolated nodes remain intact.

Files. Core: ir/incidence/projection.rs, tests/prepared_ir_backbone.rs, CORE.YAML, SYMBOLS.md, existing plan bundle. Graph: direct_ir/pagerank.rs, existing pagerank and allocation tests, scripts/performance_evidence/campaign.py and test_campaign.py, CORE.YAML, SYMBOLS.md, existing plan bundle. Reports extend the comparison families under exact-count-reduction; before-files preserve dirty work. Public extension is one defaulted trait method only; no existing signature changes.

Verification. Check default/seeded fold order and empty/signed-zero behavior, compensated accumulation versus the unchanged add recurrence on adversarial finite/nonfinite inputs, full independent PageRank oracles and directed/weighted cases. Check unit multiplicity, ignored payloads, projection reuse and warm zero allocations. Preserve tiny native parity metadata and full-scale rank hashes. Run affected debug/release correctness, Clippy, formatting, symbols and duplication.

Measurement. Extend existing Cell/Campaign selection with explicit subject filters; invalid, duplicate or empty selections fail, checkpoint and summary identity include the selected matrix. No second runner. Start with rank32 smoke; then five fresh processes for full and prepared rank on arities 2/32/1000, matched native control. Existing two warmups/nine samples, CPU 2, 180 s and 16 GiB guards, source/binary hashes, no concurrent build. Compare the preserved current baseline; record host load and keep non-equivalent preparation boundaries separate.

Risk and rollback. Numerical or allocation regression blocks adoption. If a candidate does not improve, diagnose or revert just that candidate to its before-file; never reset existing dirty work. The ordinary default owns any accepted speedup. Graph uses the recorded temporary local Core patch with Cargo.lock restored. Fresh origin/github fetch succeeded; no branch change, commit

or push. Deliver PDF, raw samples and source manifests, plus a byte-verified ReportArchive copy.

Two-incidence row specialization

Created-at: 2026-10-18 20:02 +0200. ETA 35–55 minutes: shared pair operations, numerical and callback-order tests, release gates, existing 45-cell comparison, PDF and archive. Authority: user explicitly requests the proposed repair for the two-vertex case, continuing the matched performance task. No validation or convergence weakening.

Evidence. Latest full-call medians are 178.02/41.33/30.45 ms for arities 2/32/1000, versus native hypergraphz 162.98/44.42/38.25 ms. The two-slot case visits 22 million rows over 44 iterations; prepared execution is 159.80 ms. These are timing observations, not an attributed CPU profile.

Shared Core owner. Add a defaulted IncidenceAccumulator::sum_pair hook taking two lazy term closures. Its default delegates to the policy's existing sum, preserving overridden sum behavior and coefficient evaluation order. Only IncidenceRow::gather selects this hook for a two-slot row. Unroll two-slot scatter through one shared local slot operation; keep additions sequential for duplicate targets and preserve panic/partial-write behavior. No new IR or persistent index.

PageRank policy. For a finite two-term sum, the rounded sum plus a positive zero is the same final value as the completed Neumaier two-term reduction. Preserve signed-zero behavior; nonfinite sums use the existing general policy. Test this property against the unchanged add recurrence over explicit extreme values and deterministic random IEEE bit patterns. Replace division by exactly 2.0 with multiplication by 0.5 after the same damping multiplication. All other denominators retain their existing arithmetic. The destination accumulators, full-vector/mass checks, personalization and convergence contract stay unchanged.

Scope and interfaces. Core: CORE.YAML, ir/incidence/projection.rs, tests/prepared_ir_backbone.rs, SYMBOLS.md, the existing four-format plan. Graph: CORE.YAML, direct_ir/pagerank.rs, tests/pagerank.rs as needed, SYMBOLS.md, the existing plan. Public change is an additive default trait method. No new crate, dependency, unsafe block, runtime version selector, or alternate PageRank. Use the existing campaign subject selector without changing its implementation.

Tests. Verify pair callback order and custom sum dispatch, empty and non-pair rows, negative coefficients/nonfinites, duplicate scatter targets and partial writes on a missing target. Pair arithmetic must match the established recurrence for finite results and nonfinite classification. Reuse the 432-model independent oracle, directed/weighted cases, unit storage and warm zero-allocation checks. Require tiny native parity and unchanged large-case result metadata. Run scoped debug/release tests, Clippy, fmt, symbols and duplication.

Measurements. Freeze source/binary identities after correctness gates. First use the existing selected rank smoke; then five fresh processes per cell for full HyMeKo/native and prepared HyMeKo, arities 2/32/1000. Existing two warmups, nine samples, CPU 2, 180 s and 16 GiB process guards. No concurrent compilation. A small-row speedup must not conceal a material large-row regression; retain all samples and compare against preserved exact-count-reduction evidence. If a candidate fails, diagnose or revert that candidate only before delivery.

Provenance and rollback. Fresh origin/github fetch succeeded; Core fceee42 is 2/6 and Graph 921d1d9 is 1/4 versus main, one checkout each. Preserve all existing dirty work with scoped before files. Temporary local Core Cargo patch only, restore Cargo.lock. No commit/push. Reports extend the existing families under pair-incidence, with PDF and byte-verified shared archive.

Current construction and diverse graph validation

Created-at: 2026-10-18 22:16 +0200. User requests fresh construction measurements and more diverse graphs. Preparation ETA 25–40 minutes; measurement window approximately 15–25 minutes after the user signals that the machine is free. The user's explicit 2026-10-18 reply requires waiting for that signal before performance runs. Correctness tests, compilation, provenance and runnable commands are prepared first.

Scope. Rebuild the existing canonical Core construction client against the current compact references. Compare one million vertices, 1000 disjoint 1000-member hyperedges, build and build/drop, five fresh processes per backend. Use the existing frozen native construction adapter, full checksums, 20 Criterion samples per operation, existing 1 s warmup/3 s measurement target and Core monitor. Prior processes took about 12 s, so ten repeats cost approximately two minutes. Do not run the retired manual builder; no production algorithm changes are planned.

Graph inputs. Keep the three recorded 65,536-node/one-million-incidence fixed-arity cases. Extend the existing Fixture strategy with: mixed widths [2,4,8,16,130] over 31,250 edges (one million incidences); a universal hub with 31 cyclic leaf slots per 32-member edge (one million); eight connected active regions using 32,768 vertices plus 32,768 isolated vertices (one million); a 65,536-node chain (65,535 pair edges); a smaller affine graph (8192 nodes, 4096 edges of size 32, 131,072 incidences); and a larger affine graph (262,144 nodes, 125,000 edges of size 32, four million incidences). All timed edges have at least two distinct members, within the upstream component algorithm's domain. Synthetic strategies are described precisely, not labeled real-world datasets.

One owner. Extend Fixture/Membership and a private row-descriptor helper; populate only through Core IrConstruction. Preserve the old affine RNG sequence and normalized fingerprints. Extend existing native Input reporting with actual arity/degree statistics using Core Distribution. No extra IR or adjacency implementation. Keep existing vector/residual and full component-partition checks.

Controller and schema. Add validated workload selection to the existing Cell/Campaign: omitted selection keeps the historical matrix; explicit named workloads include the new catalog. Record selection in checkpoint identity and summary expectations; explicit smoke selection is honored. Extend the existing ConstructionCampaign with explicit output/repository/backend parameters, retain its old script bytes in the new report phase and reuse the existing process monitor. No new sampler or process runner. Existing completed artifacts remain preserved.

Tests before runs. Small deterministic inputs prove mixed widths, distinct targets, seed reproducibility, hub degree, intended component/isolate counts and chain topology. Keep independent PageRank oracles and warm allocation gates. Exercise new native fixtures at small scale against both backends. Test invalid/duplicate workload selections, checkpoint identity and historical summaries. Build Core bench without timing; run –verify-builders correctness mode. Run applicable Clippy, formatting, Python lint/types, symbols and duplication gates.

Measurement protocol. Wait for the user signal; no timing probes in preparation. Inspect host load and avoid compiler overlap. First run production-scale smokes on the six new fixtures for full HyMeKo rank and native components (parity checks run both backends). Then run all nine fixtures, five existing operations, five fresh processes: 225 Graph processes/2025 samples. Existing two warmups/nine samples, CPU 2, 180 s/process and 16 GiB observed memory cap. Construction adds ten processes/400 Criterion samples. Report medians, IQR, worst values, work counts, raw samples, and host limitations. No precision change or result-dependent omission.

Files. Graph CORE.YAML, SYMBOLS.md, tests/pagerank_support/mod.rs, tests/pagerank.rs, tests/performance_evidence/native.rs, scripts/performance_evidence/ campaign.py and test_campaign.py; Core's existing report run.py; both existing plan bundles. Runtime Core/Graph algorithms and Cargo dependency pins remain untouched. Temporary local Core patch for Graph verification, restored Cargo.lock bytes.

Provenance and rollback. Fresh origin/github fetch succeeded; one checkout each, dirty work preserved. New broad-validation report phases contain before-files, task-only changes and source/binary identities. Preparation may finish with measurements explicitly pending the user. Deliver PDF readiness/results, plots and JSON/CSV, and byte-verified ReportArchive copies. No commit or push.

Virtual incidence rows and broader algorithm evidence

Created-at: 2026-10-19 02:31 +0200. Authority: user approves the proposed shared incidence improvement and broader algorithm measurements ("nezzuk"). ETA about 60–90 minutes: bounded baseline profiles, shared representation and tests, selected five-process before/after comparisons, broader algorithm clients and PDF. This is authorization to implement and measure, not a claim that all gaps are closed.

Evidence and hypothesis. The completed broad-validation campaign has nine graph shapes, exact parity, and five repeats. HyMeKo loses full and warm PageRank on hub, region and chain cases with equal iteration counts. Candidate costs are target loads, ordered compensated reductions/scatter, and vertex updates. Capture bounded samply profiles before changing the hot path. If ETW cannot start, record that concrete limitation and inspect available equivalent evidence before making any optimization claim. Never disguise a timing observation as a CPU profile.

Owner and representation. Extend Core IncidenceProjection/IncidenceRow with lossless, data-derived virtual runs only where they improve storage/work. Keep arbitrary indexed rows, ordered coefficients, duplicates, canonical IDs and immutable IR ownership. A run is derived from actual IR references, never fixture labels or seeds. No new IR, algorithm engine, dependency or global cache. Preserve the slice accessor's contract if feasible through explicit lazy expansion; hot consumers use representation-independent traversal. Avoid per-slot dynamic dispatch and large per-row overhead on pair/sparse irregular inputs. Default only an implementation with demonstrated benefit and no unexplained regressions.

Verification. Independent reconstructed targets, forward/transpose operations, nonfinite/signed-zero and compensated-order controls; empty, singleton, pair, duplicate, contiguous, strided, wrapped/irregular and large-ID cases. Reuse Core prepared-IR tests and Graph dense numerical oracles, directed/weighted policies, allocation checks and native full-vector/residual parity. Preserve existing model validation and error precedence. Core first, then Graph with temporary local Core patch and exact Cargo.lock restoration. No unrelated consumer or RL edits.

Algorithm breadth. Extend the existing typed evidence client/runner for reachability, shortest Berge path, dual reachability and selected Dijkstra/A* queries on the existing varied fixtures. All algorithms call their current owners. Use complete output/reachability/path checks against existing independent tests or mathematical invariants; no second search implementation in the measurement client. Do not invent an upstream speed ratio where BFS/direction semantics differ.

Measurement. Start with the three losing PageRank cases and a pair/32 control, then broaden only for detected risks. Five fresh processes per comparison, 2 warmups/9 calibrated samples, raw values, median/IQR/worst; keep the existing 16 GiB observed cap, CPU 2 and 180 s child timeout. Profile windows are separate from timing windows and compilation. Larger graph workloads remain bounded; avoid quadratic s-component cases without a measured/capped size. Save source/binary/PDB identities and both winning and losing results. No confirmation request is needed for the already approved Core/Graph scope; no commit/push/merge is requested.

Files and rollback. Core CORE.YAML, incidence/projection.rs and its existing test owners; Graph CORE.YAML, direct_ir/pagerank.rs only for shared-owner integration or profile-justified equivalent arithmetic, existing performance_evidence clients and controller/tests, SYMBOLS.md and the two existing plan bundles. Snapshot every touched file before modification. Revert only this task's candidate if evidence rejects it; preserve all earlier dirty work. Existing report families get an implicit-incidence phase; deliver PDF, plots, raw data and verified Samba archive.

Representation and module refinement, 2026-10-19 03:23 +0200. The first compressed implementation reduced storage but regressed timing. Profiles identified iterator and representation-dispatch overhead. Keep separate private indexed/run row types behind one IncidenceRowOperator/IncidenceRowVisitor contract; retain public slice access and canonical IR ownership. Stream contiguous/repeated runs and short boundaries only; long irregular strides retain indexed storage. Move existing target encoding/storage to incidence/projection/targets.rs and the existing Graph compensated Sum to direct_ir/pagerank/accumulator.rs, without copying implementations. New traversal evidence is a suite of the existing Campaign and ranking client, covering reachability, dual reachability, Berge BFS and unit Dijkstra.

Virtual operators and bounded planning: 2026-10-19 12:27 +02:00

Authority and ETA. User requests implementation of the proposed virtual operator algebra, bounded planning and structural/entropy-guided Berge search. Approximately 90–150 minutes: shared contracts and planner 45, Core/Graph wiring 30, correctness and report 45. No performance campaign is requested or started.

Owners. Tensor owns ordered numeric row contracts, fused row contraction, Shannon summaries and a finite branch-and-bound contraction-order planner. Core owns borrowed IR projection adapters and explicit row alignment between two IRs. Graph consumes the shared contraction for PageRank and extends its existing A* engine; no new IR, graph builder, backend, dependency, or search-engine copy.

Acceptance. Move existing numeric row contracts without arithmetic changes; shared fused gather-transform-scatter for same and explicitly mapped distinct spaces; planner validates axes and extents, retains hyperindices until their final consumer, checks integer overflow, returns a feasible plan plus budget/optimality status, and replays elementary contractions through a visitor. Work is a declared dense index-product proxy, not measured sparse time. Sparse execution may stay virtual; the planner owns only index metadata. Reassociation is explicit in the returned plan and is never silently installed in PageRank. Add goal-bound prepared Berge search using existing BFS expansion, entropy only as a tie break, and incumbent pruning in the existing A*.

Files. Tensor: CORE.YAML, hymeko_tensor/src/lib.rs, operator.rs, operator/accumulator.rs, contraction.rs, contraction/search.rs, renyi.rs, tests/virtual_operators.rs, tests/contraction_planning.rs. Core: CORE.YAML, SYMBOLS.md, src/ir/incidence.rs, src/ir/incidence/projection.rs, src/ir/incidence/contra src/tensor/mod.rs, tests/prepared_ir_backbone.rs. Graph: CORE.YAML, SYMBOLS.md, src/direct_ir/informe src/direct_ir/informed/prepared.rs, src/direct_ir/mod.rs, src/direct_ir/pagerank.rs, tests/informed_search_vv.rs. Paths under src/tests above are within the respective hymeko crate. This existing plan bundle and reports/2026-10-19-virtual-operator-planning in each owner.

Verification. Ordered/signed/duplicate/cancellation row parity; independently assembled tiny two-space products; planner versus exhaustive tiny trees, budget zero, scalar/empty extents/hyperindices/memor prepared search versus existing Dijkstra and exhaustive oracle under directed and disconnected graphs; bound pruning witness and deterministic entropy ties. Existing PageRank tests guard migration. Targeted correctness tests only, fmt, clippy, symbols and duplication gates.

Freshness and preservation. Fetched all remotes in all three owners at 12:23. Tensor main is current; Core divergence affects generators and Graph divergence only dependency pins. Existing dirty algorithm work is retained in its sole checkout; no merges, commits or pushes are part of this task. Consumer integration uses documented local Cargo patches and restores exact lock bytes; publication pins remain pending.

Budgets and risks. Search expansion limit is caller-owned and mandatory; integer operation counts are checked, no float estimate prunes. Worst case is exponential plan search and highly overlapping hyperindices. Maximum intermediate element count is a per-intermediate logical cap, not a process RSS claim. Tests use small deterministic inputs, builds use two jobs. No latency or RSS improvement is claimed without measurement. Goal-bound data borrow one immutable IR; direction restrictions can remove transitions but cannot shorten the relaxed distance. Entropy never enters a certified lower bound.

Rollback. Reports capture original dirty-file bytes before editing; restore only task hunks, retaining previous uncommitted work. Archive report and plan snapshots with the provider tool and verify bytes before completion.

Contraction runtime and cell matroids: 2026-10-19 13:29 +02:00

Authority and ETA. User explicitly accepts the proposed greedy-versus-optimized numerical measurement and requests matroids over cells. About 90 minutes: concrete execution adapter 25, exact cell matroids 25, checks/build 15, measurement/profile 15, PDF/archive 10.

Ownership and files. Extend Tensor contraction.rs/search.rs and csr_bench.rs; add contraction/matrix_chain.rs and focused test clients. Existing CSR SpMM remains the only matrix kernel. Add a maximum intermediate rank to the planning budget so a rank-two executor never receives an unsupported higher-rank product. Core keeps its existing numeric exports. Graph extends matroid/levi.rs and error.rs, adds matroid/cell.rs, exports it from matroid/mod.rs, and a cell-matroid test client. CORE.YAML entries, Core/Graph SYMBOLS and this plan bundle are updated. Reports in each owner: 2026-10-19-contraction-runtime-cell-matroids.

Numerical acceptance. Compile the same matrix-chain problem and existing plan into reusable buffers. Warm execution uses the existing serial SpMM; plan and preparation costs are separate. Inputs, shape, dtype and output semantics match between greedy and optimized. Closed-form integer cases and nonuniform signed cases check reassociation error. A rank-two backend is an explicit specialization, not a general new tensor backend.

Matroid acceptance (user clarification, 13:41). Levi connectivity over incidences, not a linear column matroid. The user explicitly selected this meaning. Extend the existing Levi source traversal and shared union-find so the same owner accepts canonical IR and graded CellTopology views. Ground elements are exact (rank, cell, slot) incidences. Return rank, components, spanning basis, independent-set checks and incremental offers. Include all ranks and isolated cells, preserve duplicate incidences, and do not materialize a Levi graph. Verify rank = total cells - components, heredity/exchange, incremental/batch parity and existing IR Levi regressions. The earlier exploratory exact-field files are removed from the deliverable.

Measurement. Criterion existing csr_bench opt-in target, at least five fresh processes, per-process samples/median/IQR/worst retained. Two matrix scales, same declared serial kernel, f64, CPU 2, 16 GiB process peak cap and 180 s process timeout. Record seed, input dimensions, source/binary hashes, planning states and host activity. Compare warm execution and full planning/preparation/one execution. Reuse the existing ProcessRunner. Samply attribution attempted after compilation; capture failure precisely if ETW startup remains unavailable. No timing overlaps builds or matroid tests. No claimed HyMeKo-wide or HypergraphZ speedup from a matrix-chain example.

Risks and rollback. Planning cost may dominate small/one-shot calls; sparse intermediates can densify; changed association can move floating values. Levi bases certify connectivity only, not numerical equivalence; no constant-time/rank-free claim. Preserve previous dirty changes and snapshots; no commit, push or merge requested. Fetch of all remotes succeeded for Core/Graph/Tensor at task start; existing divergence and pins are unchanged. CLI path patches are local integration aids, lockfile bytes restored. Verify and archive the PDF and raw evidence before completion.

PageRank fused state sweep: 2026-10-19 15:59 +02:00

Authority and ETA. User asks to increase the existing advantage. About 45–75 minutes: shared accumulator operation and workspace lifecycle 20, correctness/gates 15, PDF/archive 10, optional explicitly requested measurement extra. This request authorizes implementation; new benchmark/profile execution awaits an explicit current-task request.

Evidence and owner. Existing chain profile attributes 1078/3260 samples to finish-update and 306 to input update; hub/region profiles show compensated gather/scatter. The scalar recurrence remains current; use this evidence to eliminate redundant whole-vector passes without weakening precision. Graph owns PageRank iteration state. Tensor owns the additive ordered accumulator take-and-reset primitive. Core IR and construction unchanged.

Change. Move the existing PageRankWorkspace into pagerank/workspace.rs. Initialize scaled input and zero accumulators together once. Each iteration consumes and resets incoming accumulators while validating scores, accumulating mass/L1 change and staging the next normalized input/dangling mass. Preserve per-accumulator operation order and exact scores, convergence and errors. Initial staging plus one per-step vertex sweep replaces the separate normalization, clearing and finishing sweeps. Final staging may be unused at early convergence; assess that overhead rather than assuming a speedup. No new IR or algorithm engine.

Files and interfaces. Tensor CORE.YAML, src/operator/accumulator.rs and existing virtual-operator tests; Graph CORE.YAML, SYMBOLS.md, src/direct_ir/pagerank.rs, pagerank/workspace.rs, private workspace regression tests, existing pagerank.rs/alloc tests. Public PageRank signatures remain unchanged; OrderedAccumulator gains a default take_total. Retain frozen before bytes and optional baseline executable. Existing plan four formats; reports/2026-10-19-pagerank-fused-state under affected owners.

Correctness. Frozen pre-change sweep oracle checks scores, scaled input, dangling sum, L1 and mass decisions bitwise; include weighted/unweighted, isolated, uniform and personalized, empty, error and repeated-workspace cases. Existing independent dense PageRank oracle and all direction/weight/allocation gates remain. No approximate summation, changed tolerance, unsafe code, dependency, or new graph materialization.

Measurements and risks. Prepare a baseline executable but do not run timing without explicit authorization. If requested, use existing native evidence client on hub, regions, chain and pair control, five fresh processes, same sources/configuration/affinity, raw medians/IQR/worst, full vector checks and snapshot hashes. Observe 16 GiB/180 s limits; report Samply's known startup limitation if a requested profile cannot be captured. Fused lifetime may expose stale state after an error; initialization must reset every live entry. Scalar arithmetic order must remain exact. Do not claim a runtime gain from fewer source loops alone. Preserve existing dirty work; no commit/push requested.

Control approval, 2026-10-19 16:15 +02:00. The user explicitly approved five fresh processes on hub32_1m, components8_1m, chain65k and rank2_1m, comparing captured before/after HyMeKo binaries and the pinned Hypergraphz native DLL. The existing ProcessRunner and evidence parser own execution and validation. Rotate order, pin CPU 2, retain nine samples per process, enforce 16 GiB / 180 seconds and 15 seconds per operation. No build or profiler overlaps measurement. First production-scale cell is the smoke.

Structural PageRank execution: 2026-10-19 19:10 +02:00

Authority and ETA. User explicitly requests structural/BnB integration and fixing profile-identified costs to improve against Hypergraphz. Estimated 60–90 minutes: shared planner/range primitives 25, adapters and guarded iteration 25, correctness/gates 20, PDF/archive 10. No commit/push requested. Current-task timing/profile execution requires explicit confirmation before launch; prepare concrete tested candidate first.

Owners and change. Tensor: factor the existing bounded search into one generic branch-and-bound owner consumed by contraction-order planning and structural execution planning. Add range contraction to the existing sparse operator module: compensated prefix sums for range gather, difference endpoints plus prefix accumulation for transpose scatter. Core: expose lossless range visits/counts over the existing projection, preserving multiplicity; no expanded tensor or second IR. Graph: reuse existing union-find for projection component labels; build a bounded component-aware direct/range execution plan. Exact topology and spans determine feasibility; modeled integer work determines pruning; heuristic ordering never proves numerical equivalence or timing superiority.

Numerics. Only unit undirected routes initially admit range regrouping. Weighted and directed policies retain the existing canonical contraction. Keep compensated sums, reduce their number algebraically. Range subtraction can lose precision: require a canonical compensated update before accepting convergence and at the iteration limit. On invalid fast state restore the prior scores and perform the canonical step. No relaxed tolerance, mass check, negative-value clamp, discarded incidence or approximate graph. Reassociation may change bits; independent dense and high-precision references govern acceptance.

Files. Tensor contraction/search.rs and reusable search module, operator/ranges.rs, operator/accumulator.rs; Core incidence/projection.rs and its targets adapter; Graph components.rs, matroid/workspace.rs internal visibility, pagerank plan and workspace. Extend existing targeted tests and add range/planner correctness fixtures under owners. CORE.YAML records this explicit grant; SYMBOLS regenerated. No dependencies or persistent path patches; restore lock bytes after local Core/Tensor integration commands.

Verification. Existing contraction subset-DP optimum oracle must still pass. Generated bounded-search cases, range dense-oracle checks (empty, duplicate, reversed, discontinuous, signed and large-dynamic-range), component partition parity, numerical PageRank oracle with nonuniform personalization, isolates, mixed compressibility, directions, weights, tight tolerance, cap and fallback. Warm allocations remain zero. Small direct preparation retains its existing allocation contract. Freeze current binary/source before edits.

Performance scope if authorized. Existing ProcessRunner/native client, hub, regions, chain and pair control, five fresh processes and nine samples, same boundary and native DLL. No build/test overlaps timing; record host activity, source/binary hashes and correctness/residuals. Process caps 16 GiB/180 s and operation cap 15 s. New profiler startup must be bounded and diagnostics retained; do not claim causal attribution on empty output.

Risks. Prefix cancellation, endpoint drift, component gaps, setup overhead, search budget, sparse row multiplicity, personalized dangling correction. Numerical fallback uses the same owner and counts within the iteration budget. Plan costs are nominal work, not runtime bounds. Preserve all prior dirty work. Final PDF plus raw evidence archived and verified byte-for-byte in the shared report store.

Measurement approval and profile refinement, 19:30. User explicitly approved five-process matched control and Samply profiling at 19:23. First control completed 60/60 cells with parity. Fresh hub profile captured and resolved (5034 thread samples); range closures account for 1972 samples. The encoded hub-to-first-leaf jump splits the following contiguous run: 93764 ranges for 31250 rows. Refine the same TargetRun encoder to emit a singleton before a contiguous run, preserving canonical slot order and multiplicity. Add exact reconstruction/count tests; rerun the affected four-case control against the intermediate binary and native DLL. Keep both campaigns; no profile/build overlaps timing.

Degree-bound refinement, 19:38. The four-case controls leave a small chain latency gap. For unit undirected input, the prepared incidence count at each destination certifies the maximum number of additions. If at most two finite nonnegative terms arrive, ordinary accumulation has the same final bits as the existing proved Neumaier two-term specialization. Separate gather and scatter accumulator policies in the same RowContraction: keep compensated gather for arbitrary row widths, use scalar scatter only under the exact per-destination bound. Reuse the workspace auxiliary buffer; transfer its completed values through the same state sweep. Add bitwise generated parity tests against canonical scatter, retain cap/mass/allocation checks and rerun the four controls. No approximate summation policy.

Construction commit and reference streaming: 2026-10-19 20:34 +02:00

Authority, scope and ETA. User asks to pursue the remaining construction and memory gap. About 40–60 minutes: evidence/baseline 10, common-owner implementation 15, correctness and consumer checks 15, PDF/archive 10. Current benchmark/profile execution awaits explicit authorization; the frozen baseline and static layout are prepared first. Core owns this change. Preserve the existing IR, public record fields, serialization, source provenance, facets, namespace roots, snapshots and typed ID domain. No dependencies, commit, push, or unrelated consumer migration.

Evidence. Latest canonical construction control: 68.85 ms build / 82.23 ms build+drop, versus native 59.85 / 62.79 ms; process peaks 176.56 / 69.21 MiB. One million nodes, 1000 disjoint hyperedges of 1000 members. Old October 11 profiles used the retired manual builder and cannot attribute the present cost. Source shows placeholder declaration and NONE-dispatch writes immediately overwritten by final data, and a materialized draft reference Vec before compact storage. Static layout will identify retained record costs; it is not a new timing/RSS measurement.

Implementation. Extend Ir's existing declaration append with a private encoded slot input; public raw append retains NONE semantics. IrConstruction commits the final record once for appended declarations and replaces the existing placeholder for indexed lowering, through one private publication method. Preserve shared named-record/link logic. Add an iterator input method for hyperarcs; existing Vec entry delegates to it. Use existing store_references, with the synthetic canonical input adapter supplying a lazy iterator. All typed checks precede iterator consumption. Also reject inconsistent parallel declaration storage before append_arc mutation, matching its documented construction guarantee.

Files. CORE.YAML, SYMBOLS.md, hymeko_core/src/ir/ir.rs, existing reference storage only if demonstrated necessary, benches/ir_bench.rs and ir_comparison.rs; tests in the existing IR unit-test owner. Existing plan bundle and reports/2026-10-19-construction-commit. Public API is additive; no AnnoR/NodeRec layout migration in this step. Report the measured layout and concrete migration impact rather than silently changing public value types.

Verification. Wire equality of appended and indexed construction, provenance, facets, hierarchy/namespace/backlinks, reference sign/direction/payload and exact capacities. Invalid owner/parent/parallel tables return before consuming iterators or mutating IR. Published snapshots remain isolated. Existing compiler/CBOR/builder and Graph consumer correctness checks; Clippy, fmt, symbols and duplication with unchanged baselines.

Requested measurement plan. If authorized: current baseline Samply profile, then matched before/after/native construction and destruction, five fresh processes, 20 Criterion samples per phase, CPU 2, existing ProcessRunner/sampler/DLL. Capture source, binary, commands, membership checksum, host activity, process peak RSS and raw samples. 16 GiB and 180 seconds per process. No builds/profiles overlap timing. Final profile uses matching PDB and offline symbol resolution. Public conclusions stay workload-specific.

Risks. Parent links and optional side-table lengths must be extended exactly once; indexed lowering must still replace its reserved slot; empty and failing streams must not create partial declarations. Iterator panics are distinct from checked construction errors. Memory footprint remains constrained by public inline record fields. Preserve earlier changes, compare only captured current binaries, and archive PDF/evidence with byte checks.

Measurement approval and baseline, 20:38. User explicitly authorized current construction profiling and five-process before/after/native controls. The matching debug-info baseline profile completed in 12.24 seconds, 6694 thread samples, 4964 mapped to Rust leaves. Record writes, construction commit, parent checks and draft-vector destruction are visible. Both candidates use profile.bench.debug=1 and the same local Tensor patch; locks are restored. Static x64 layout: DeclNode 96, NodeRec 32, EdgeRec 56, ArcRec 80, AnnoR 48, ReferenceDraft 96 and SignedRefR 32 bytes. These sizes are not peak RSS measurements.

Validated construction scope refinement, 20:57. The first matched control reduced construction time but retained a small native gap. The matching after-profile contains 650/7814 leaf samples in check_parent, plus 179 in the empty-Vec in-place reference path and 157 in draft destruction. IrConstruction's exclusive borrow permits one validated parent cursor (ID and last child), updated only by the shared link owner. NONE parents remain an immediate no-op; parent changes revalidate. No external mutable IR reference escapes the capability. Keep iterator/error checks; cache invalid inputs never. Skip empty concrete base Vec conversion, still dropping any reserved input allocation. Verify alternating parents, mixed node/edge/arc writes, failed admissions and new-scope revalidation; preserve the intermediate binary/source and repeat matched controls.

Compact canonical payloads: 2026-10-20 01:46 +02:00

Authority and ETA. User approves compact annotation/base-list storage. Core owns storage; Graph and Transformation consume it after Core gates. Estimated 60–100 minutes for storage, migrations, correctness and PDF/archive. No benchmark, dependency, commit or push. Preserve the captured initial dirty sources. Fetched Core is 2 ahead/6 behind main (generators); Graph 1 ahead/4 behind (pins); Transformation 2 ahead/0 behind. No merge, worktree or source reset in this scope.

Design. Extend existing AnnotationId/storage to declarations and arcs; add typed BaseListId and one IrTable of nonempty canonical reference lists. NONE denotes empty without a slot. AnnoR remains owned input; DeclNode's payload parameter separates owned construction/wire input from IDs using one field schema and one IrConstruction. Preserve logical serde via the existing owner-aware adapter. Borrowed lookups/indexing resolve local IDs. No nonempty content interning or ID narrowing.

Mutation. Declaration/arc annotations own distinct nonempty slots; reference payload sharing remains supported. Admission rejects aliased editable slots. Value edits replace slots under IrTable COW; first use allocates, repeated edits do not accumulate old payloads. The existing symbol visitor handles shared reference payloads once and retains incremental counts. Preserve weights, float bits and order.

Files and checks. Core ids, IR/reference/wire/access/coherence/digest, resolution/symbols, value-update/CBOR consumers and existing tests/bench source clients. Graph/Transformation IR readers and fixtures only. No RL or unrelated work. Extend CORE.YAML and this plan. Test 56/16/40/40-byte layouts, old wire/digests, invalid IDs, empty/filled payloads, snapshots, repeated edits, symbol compaction, base multiplicity. Run inspected correctness targets, check/clippy, fmt, symbols and duplication. CLI-only dependency patches; restore locks. No timing/RSS run.

Risk and rollback. Dense metadata still costs payload storage; COW may copy tables. Revert only task hunks from the captured inputs. Extend construction evidence with a compact-storage PDF and verified Samba archive. Static layout savings are distinct from unmeasured speed and whole-process memory effects.

Storage refinement, 2026-10-20 02:07. Nonempty base lists use exact boxed slices rather than retained Vec headers: the 8-byte ID plus 16-byte slice header equals the former 24-byte Vec header on x64, while empty lists cost only an ID. The 48-byte annotation payload stays unchanged; a fully populated annotation pays its additional 8-byte ID. Declaration serialization is restricted to logical annotations (owned or borrowed), so standalone compact IDs cannot silently enter logical wire output. The existing Ir serializer resolves them.

Compact storage control: 2026-10-20 02:45 +02:00

Authority and ETA. User explicitly approves the proposed construction/drop/RSS, metadata-density and reading/value-update control. About 30–45 minutes including preparation, two optimized builds, 12–18 minutes of timing, analysis and PDF/archive. No profiling, production-library change, dependency update, commit or push in this phase.

Owner and files. Extend Core's existing SyntheticIrBuilder and Criterion clients: benches/ir_bench.rs, its private ir_bench/metadata.rs module, interning_bench.rs; existing ir_comparison.rs remains the native adapter. Extend the existing report control.py with configured Criterion cells, reusing Graph's ProcessRunner and source capture. CORE.YAML records approval; reuse this four-format plan and compact/control evidence folder.

Before/after identity. The preserved pre-compaction source is the baseline. Build an instrumentation-only evidence copy; never reset or swap the live Core checkout. Both use identical benchmark inputs, two explicit read-seam substitutions for old inline annotation/base fields, the same local Tensor source and profile.bench.debug=1. Preserve unmodified historical sources/binary. Record every source/binary hash and benchmark edit. There is one active canonical IR and builder, not a second maintained implementation.

Cells. Five fresh processes per subject, rotated order, 20 Criterion samples, 1-second warmup and 3-second sampling windows. Native topology: 1 million vertices, 1000 disjoint 1000-member hyperedges; before/after/Hypergraphz, build and build+drop. Metadata: the same shape, node annotation plus one root base on none, every 100th, or every node; before/after build, build+drop and full annotation/base scan. Reference sign and vector semantics remain unchanged. Expected counts and checksums are independently checked before timing. Native comparison covers only the shared topology. Value updates reuse the existing 20000-node source fixture, checked numeric/text updates, flat and 256-record paged storage; preparation and result drop stay outside update timing. A dedicated opt-in dispatch excludes unrelated benchmark setup from peak-memory evidence.

Budgets and gates. Reuse CPU 2 affinity, one Rayon thread, ProcessRunner 16 GiB / 180 seconds per process and 15 seconds per normalized operation. No build or profiler overlaps timing. First run of each cell class is the production-sized smoke; reconcile large runtime deviations before continuing. Save raw samples, median/IQR/worst, checksums, semantic hashes, source identity and host CPU activity. Persistent unrelated processes are observed, never killed. Process peak is not retained graph allocation.

Verification, risk and rollback. Small density fixtures and deliberately wrong expected values must distinguish bad results; full membership verification remains. Unchanged semantic digests must match across binaries. Default-off switches guard new timing. Clippy/fmt/symbol/duplication gates cover benchmark changes. Restore Cargo.lock exactly; revert only current-task benchmark hunks if necessary. Dense payloads may cost more, new indirection may affect scans, process startup/allocator and host activity may confound RSS. Record negative outcomes without a universal speed claim. PDF, numerical data and plots are archived through the shared provider with byte verification.

Control outcome, 2026-10-20 03:29. All 55 processes / 3200 samples completed, with fixture/hash parity and an unchanged source manifest. Current matched build+drop medians: before 69.692 ms, after 58.390 ms, native 72.607 ms; peak working sets 175.35 / 121.86 / 69.97 MiB. Dense metadata costs 12.1 / 10.9 / 18.5 percent more for build / build+drop / read. Host CPU

median 20.3 percent, maximum 51.7 percent: this is preliminary, not quiet-host confirmation. User explicitly selects the preliminary report for now; no further measurements or profiling. Preserve original measured driver before post-run Python typing/style cleanup. PDF, source/binary snapshots and raw data are archived and byte-verified; no production library edits or publication in this phase.

Dense base-list reuse: 2026-10-20 03:43 +02:00

Authority and ETA. User asks to fix the dense metadata regression overnight. Core implementation and correctness are authorized; focused profiling/control permission is pending. About 30–50 minutes for code, tests and PDF; measurement time separately. Core fetched at 03:36, Tensor dependency refreshed; no branch switch, merge or publication.

Discovery and ownership. Existing Ir::store_bases owns immutable lists; IrConstruction::commit_named and wire NodeInput/EdgeInput feed this storage. Reference payload sharing is already admitted and symbol-rewrite visitors visit shared payloads once. Declaration and arc annotations retain exclusive editable slots. No new public API, canonical model, builder, dependency, persistent cache or standalone report family.

Design. Reuse the most recently stored nonempty base list when the entire incoming list matches. Compare owned drafts against borrowed canonical payloads before converting references, shrinking vectors or allocating annotation/weight entries. Empty input remains NONE. Raw canonical references use exact handle equality; draft comparisons resolve current payloads, compare tags/order, sign/direction/target, optional values and weights with ValueR::bitwise_eq. This distinguishes signed zero, NaN payloads and absent versus empty vectors. Reuse survives empty lists and new construction scopes, but is bounded to one candidate, not global interning. Unique lists use the existing allocation path.

Scope and risks. Extend reference.rs, commit_named and existing wire adapter; add discriminating tests in reference/tests.rs. No pooling of editable declaration annotations. Worst-case repeated-prefix comparison adds linear work on a miss; no unbounded search. Table capacity growth and arbitrary distinct-list allocations are not addressed by this change. The existing dense benchmark repeats one plain base list one million times. Structural allocation elimination does not prove a wall-time improvement; previous timings were nonquiet.

Verification. Check all signs/directions, multiplicity/order, targets, tags, float bits, optional weights, invalid IDs, empty/nonconsecutive lists, COW snapshots, failed admission, symbol compaction, logical JSON/CBOR/hash parity and wire reuse. Run focused Core tests first, existing Core correctness targets, clippy/fmt/symbols/duplication. Use the local Tensor CLI patch and restore Cargo.lock exactly. Preserve initial dirty files; rollback only current task hunks. PDF/evidence archive with byte verification, no commit/push.

Verified implementation, 2026-10-20. Core library 268 and unified IR integration 181 tests passed; nine ignored tests were not enabled. All-target workspace Clippy and fmt/symbol/duplication gates passed. The optimized benchmark ran only its 20 small construction verification cases. Four source files differ from the prior measured candidate, with unchanged benchmark source. The repeated-list algorithm retains one canonical list instead of N; for the existing million-singleton input, 47,999,952 logical bytes of references/headers are avoided. This is not RSS or an allocator-call measurement: input drafts still allocate. Profiling/control permission has not arrived; runtime improvement is not claimed. Lock restored; no commit/push.

Dense reuse matched control: 2026-10-20 12:46 +02:00

Authority and scope. User explicitly approves dense-case profiling and empty/dense before-after controls. Five repetitions, about 8–12 minutes of measurement and profiling, plus evidence/PDF. Use the two already-built matching debug-info binaries; all 308 source files and executable hashes were reverified. No production changes, new graph representation, dependency update, commit or push. Fresh fetches succeeded: Core remains 2 ahead/6 behind main; Tensor matches main; Graph runner checkout 1 ahead/4 behind.

Discovery. Existing ConstructionControl, CriterionSummary and ProcessRunner own configured measurements, parsing, affinity and limits. Extend their hardcoded campaign counts to derive from the existing config, retain five repetitions and twenty samples. Reuse ConstructionProfile/LLVM-PDB resolution, existing input fixture, four-format plan, and compact/dense-reuse/control evidence directory. Native library is not needed or measured.

Controls. Two metadata densities, before and after, five fresh processes each: 20 processes, three phases (build, build+drop, read), 1200 normalized Criterion samples. One million vertices; 1000 disjoint edges of 1000 members; dense adds one tag, numeric value and identical positive root-base reference per vertex. No source/parser/IO timing. Rotate binary order per repetition, CPU 2, one Rayon thread, 1-second warmup/3-second sample window per phase, existing 16 GiB and 180-second per-process limits and 15-second normalized-operation cap. First complete repetition is the smoke gate; resume only after fixture identity, budgets and plausible time are checked.

Profiling. After controls finish, sample dense build in each binary using Samply save-only, rate 1000 Hz, Criterion profile-time 8 seconds; cap profiler process at 60 seconds. Capture matching executable/PDB, ETL and compressed profile, resolve own-code leaf addresses offline. Sampling includes setup/drop and allocator activity; it is explanatory evidence, not a replacement for separated Criterion phases.

Evidence and checks. Preserve both binary/source identities, raw samples, fixture checksum, command/config/environment, host CPU activity and process snapshots. Cross-check all sixty phase medians against Criterion estimates. Existing 449 correctness passes are current; repeat only small binary verification probes. Validate dynamic config counts against prior captured control data, lint changed Python, no performance parallelism. Report median/IQR/worst, process peak scope, quiet-host limitations and profile attribution coverage. PDF and complete evidence use the shared archive with byte verification.

Control outcome, 2026-10-20 13:00 +02:00. Twenty fresh processes, 1200 normalized samples and sixty cross-checked phase medians passed; all source hashes remained unchanged. Dense medians before/after: build 184.426/128.919 ms, build+drop 226.724/147.854 ms, read 11.787/6.741 ms. Maximum process peaks 245.19/183.43 MiB. Each of five paired dense repetitions improved in all phases. Empty medians changed by +0.7/+3.1/+0.2 percent, with overlapping IQRs. Host activity median 19.3 percent/max 39.0 percent; persistent external Python PID 44840 accumulated 752.75 CPU seconds. This was not a quiet-host campaign. Two separate Samply profiles completed in 15.54/27.17 seconds. Own-binary leaf coverage was 2114/7570 and 2809/8147 weighted samples. store_reference leaves fell 163 to 31; sample counts are not operation timings or allocator-call counts. No production source/dependency changes, commit or push. PDF and raw evidence are archived with content verification; retain both source and binary identities.

Bounded PageRank finish: 2026-10-20 13:22 +02:00

Authority and ETA. User requests strengthening the Hypergraphz advantage, then explicitly approves five-repeat chain, two-member, hub and region controls against before/native. Roughly 25–40 minutes implementation/checks, plus 5–8 minutes measurement and PDF/archive. No new profile, dependency, commit or push. Fresh fetches completed: Core 2 ahead/6 behind main, Graph 1 ahead/4 behind, Tensor matches main; preserve dirty work.

Discovery. Existing chain profile assigns 1702/3508 thread samples to finish_update and 1173 to PageRank::build. Bounded scatter currently materializes every f64 result as Sum in incoming, clearing auxiliary, before finish_update immediately reads/resets incoming. Graph owns this PageRank-specific update; reuse Tensor's existing OrderedAccumulator trait. Core IR, projection and operator are unchanged.

Implementation. Move the single vertex update loop into a private borrowed VertexUpdate owner. Existing compensated and bounded-scalar wrappers call the same generic loop. For scalar totals retain exactly Sum::from_first(value).total() in registers, preserving signed-zero/nonfinite behavior without materializing the intermediate Sum array. Keep contribution order, compensation of change/mass/dangling, failure gates, score writes, teleportation and convergence. Clear scalar scratch on error; subsequent runs remain reusable. Only Graph pagerank.rs, workspace.rs and existing workspace/tests.rs need source edits.

Proof and risks. Compare scalar and old two-stage finishing bitwise, including zero signs, subnormals, different personalization/degree distributions, nonfinite/error recovery and capacity reuse. Existing generated degree-bound tests compare complete rank vectors and convergence with the canonical compensated path. Run targeted unit, integration, release numerical and allocation gates before benchmarking. No scalar accumulator is introduced where the existing incoming-term bound does not apply. Compiler/code-layout and host activity may erase or reverse a small speedup; report all outcomes, no forced win.

Matched measurement. Build before from current Core/Tensor/Graph sources, then after with identical CLI patches and flags. Restore lock bytes after each command. Capture source and executable/PDB hashes. Extend the existing structural control/config owner, ProcessRunner and Evidence parser; no copied runner. Four fixed workloads, three subjects, five rotated-order processes: sixty cells, nine timed samples each, two warmups. CPU 2, one Rayon thread, 16 GiB/180 seconds per process and 15 seconds per operation. Compare full outputs, fingerprints and iteration counts; before/after must agree bitwise. First production-sized cell is the smoke gate. No compilation/profiling overlaps timing.

Delivery. Reuse the four-format plan, structural report family, shared archive and byte verification. Preserve initial source and task-only diff. Produce PDF with raw sample provenance, median/IQR/worst and host activity. No universal ranking or memory claim.

Rejected fusion, 2026-10-20 13:36. The full sixty-cell control passed bitwise rank/iteration parity, but chain median was 21.123 ms before and 21.566 ms after. This does not secure the target advantage. Restore only the three current-task source files from the verified initial snapshot; keep candidate binary, source and data. The next isolated change shares the existing exact divide-by-two implementation between PageRank::fraction (destination normalization) and InputNormalization::scale (source normalization). Weighted/zero-degree behavior stays unchanged; other divisors keep division. Move that arithmetic into InputNormalization::divide and call it from both existing owners; no new IR or loop. Test original division versus shared normalization on IEEE

extremes and 100000 seeded patterns, retaining error, convergence and warm-allocation checks. Repeat the same approved four-workload matched control with the original baseline and a fresh candidate. This extends the measurement window by roughly five minutes; no fresh profiling is added.

Final local result, 2026-10-20. The shared exact-normalization change is retained; the fusion implementation is absent from live source. Final controls: chain 24.539/23.214/26.642 ms, pair 126.138/128.028/181.105 ms, hub 129.659/127.509/149.763 ms, regions 91.772/92.902/138.201 ms (before/after/native). All twenty paired native comparisons favored HyMeKo. Own before/after effects are mixed: -5.4/+1.5/-1.7/+1.2 percent; overlapping IQRs. The host was busy (median 25.8 percent, maximum 62.3 percent), so incremental speed claims remain preliminary and require quiet-host confirmation. No samples were dropped. Thirty-three final targeted tests passed, including 100000 seeded IEEE patterns and zero-allocation warm solves. Both sixty-cell campaigns preserved bitwise HyMeKo vectors and iteration counts; 1080 raw samples and 120 cell medians were independently checked. Static gates passed with unchanged duplication baseline. Both candidate histories are retained; no production Core/Tensor edits or new profiler. PDF, binaries/PDB, source snapshots and raw records use the verified shared archive.

NiTech kato21 control: 2026-10-20 15:00 +02:00

Authority and environment. User supplies kato21 access for benchmarking, requires not disturbing other users, and explicitly permits offline edits against the verified current Core/Graph/Tensor source state. VPN blocks origin 192.168.1.228:2222; GitHub fetches succeeded. No merge, branch change, new checkout or publication. Authenticated host: Ubuntu 24.04.3, x86-64, tcsh, Threadripper 7980X (64 cores/128 threads), 125 GiB RAM, 96 GiB available, 1 NUMA node, CPU 2 sibling 66. User home is NFS; use a private local /tmp directory on NVMe. Root graphical session and hajdu login were present; no other human users appeared. Home README requires nice on long jobs.

Scope and ETA. Prepare and verify a Linux matched comparison using the same four PageRank fixtures, two HyMeKo versions and pinned native Hypergraphz. Allow roughly 30–60 minutes for portable adapter, transfer/toolchain/cold single-core build and gates, then 5–10 minutes for timing, plus report. Stop before the next cell if another user logs in or background/core activity indicates competing work. No new algorithm, IR, runtime dependency, profiler or GPU work.

Discovery/owners. Extend Graph's existing native/library.rs loader with Linux dlopen/dlsym, and its telemetry module with process CPU and /proc memory data. Enable the existing native client on Linux. ProcessRunner already owns Linux affinity, RSS and limits; retain it. Extend its source verifier for explicit immutable unpacked snapshots without git metadata, and add tested quiet-host inspection. Existing StructuralControl selects platform/CPU and applies the quiet guard; no copied runner. Core measurement and canonical IR remain the algorithm/data owners.

Version set and transfer. Preserve local current source manifest and create an exact source archive plus Cargo-vendored dependencies from the offline cache. Never send credentials or SSH keys. Core and Tensor are identical in both builds; baseline uses only the three pre-normalization Graph files from the captured task input. Record all transformed build-only dependency configuration; restore Cargo.lock. Use Rust 1.97.1 in a task-local toolchain (existing user 1.96.1 remains unchanged). Cross-compile the pinned Hypergraphz adapter for Linux with the existing Zig 0.17.0-dev.242 compiler, ReleaseFast and baseline CPU, preserving upstream source.

Resource and correctness gates. Single build worker; nice 10; inherited CPU 2 affinity; per-task memory cap 16 GiB. Builds and tests use local scratch. No timed workloads overlap builds. Use owned transient user service and logs for long work, with elapsed status, no interactive windows or root changes. Run platform/helper tests and small native parity, then production-sized smoke per workload class before scaling. Require full rank/iteration/fingerprint parity and unchanged input/source identity. Existing Windows compatibility checks remain required.

Measurement and evidence. Four workloads (chain, pair, hub, regions), three subjects, five rotated-order fresh processes: sixty cells, two warmups/nine normalized samples each. Same damping/tolerance and mathematical scope as the local controls. ProcessRunner enforces 16 GiB/180 seconds and analysis checks 15 seconds per operation. Record tools, binaries, source/dependencies, topology, host sessions/load, priority, affinity, wall/CPU, process peaks, raw samples and complete errors. PDF with cautious cross-host interpretation. Archive through the shared provider when VPN routing allows; retain local complete evidence and report any unavailable archive honestly.

Artifact audit correction, 2026-10-20 16:21 +02:00. The completed first Linux campaign used identical before/after binary hashes. Cargo marked the baseline fresh in the shared target

directory, despite different snapshot source paths. Thus the A/B labels cannot support a normalization decision; retain all records and do not revert production source on that evidence. Extend the task-local LinuxBuild owner to use separate target directories per label, record a forced cold baseline build and distinct binary identity, then rerun the same sixty-cell approved control in a new evidence directory. Reuse the verified fresh candidate artifact. Keep first campaign logs immutable. Add about 10–15 minutes for isolated compilation, parity, quiet-host control and audit. No new algorithm or profile. Require compiler-artifact fresh=false for baseline, separate output roots, unchanged source hashes and full rank-vector parity before measurement. Preserve both campaigns.

Verified decision, 2026-10-20 16:34 +02:00. The corrected sixty-cell campaign has distinct binaries and all parity/source gates pass. Before/candidate/native medians in ms: chain 19.648/20.460/24.139, pair 124.994/125.916/164.738, hub 106.277/107.664/130.491, regions 70.631/71.928/118.7
The candidate adds 4.13/0.74/1.30/1.84 percent. Restore only the three owned input-normalization files to the measured baseline, after proving live contents still match the captured candidate. Retain every other Core/Graph/Tensor improvement and the portable measurement adapter. Run the 32 baseline PageRank correctness checks, Clippy/fmt/index/duplication gates; record that live algorithm inputs match the measured baseline. Report one selected variant for all workloads and archive both the invalid-label and corrected campaigns.

NiTech vertex-permutation control: 2026-10-20 16:49 +02:00

Authority and ETA. User explicitly approves the proposed same-graph vertex renumbering experiment on kato21. Prepare about 15–25 minutes, then approximately 15–20 minutes of guarded timing plus PDF/archive. Fresh origin/GitHub fetches pass; keep current dirty owner checkouts (Core 2 ahead/6 behind; Graph 1/4; Tensor 0/0). No production algorithm, IR model, dependency, profiler, Git branch or publication change.

Owner and discovery. Extend Graph's existing test Fixture with an optional seeded vertex permutation; keep its one IrConstruction population path. Existing small rank permutation test is an oracle pattern, not a production renumbering owner. Extend native Input with exact incidence and inverse-mapped rank validation. Record projection ranges and selected plan. Reuse StructuralControl, ProcessRunner, QuietHostGuard and the existing NiTech LinuxBuild/ControlSession by adding configuration, not copied runners. Affected source: pagerank_support/mod.rs, pagerank.rs, performance_evidence/native.rs, performance_evidence/ranking.rs; existing report control.py/build.py/run_control.py. CORE.YAML records approval; SYMBOLS is regenerated. Dependencies unchanged.

Invariant. Original-to-stored permutation is a bijection; only targets are mapped, preserving arc order, member order, counts, signs and neutral unit weights. All IR creation still calls the canonical Core builder. Original seed remains 335597; permutation seeds 17, 29, 43 use deterministic Fisher–Yates with rejection sampling. Validate exact inverse-mapped incidence streams and degrees, plus full rank L1 delta at most 2e-10. Numerical reduction order may change: do not require bit parity between layouts, but require bit parity for repetitions of one layout. Uniform personalization is unchanged by a permutation; no directed/weighted speed claim in this experiment.

Matrix and resources. Four existing fixtures (chain, pair, hub, regions), identity plus three permutations, HyMeKo/native, five fresh-process repeats: 160 cells, nine timed samples/two warmups each. First repetition is production-scale smoke for all layouts before the remaining repeats. Same total PageRank-call timing boundary; creation, permutation, isomorphism and native parity are outside timing. Rotate order. Same CPU 2 / SMT sibling 66, nice 10, idle I/O, 16 GiB, 180 seconds per process, 15 seconds per operation. Guard other users/load before every new cell. Pause on busy. Toolchains and vendor remain in owned local NVMe scratch; NFS home unchanged.

Gates and evidence. Unit-test bijection, deterministic seed behavior, identity, malformed seed rejection, topology preservation and mapped rank agreement. Run Windows targeted clients/Clippy and Linux clients/native parity before smoke. Snapshot current sources and compile into a unique target directory; archive compiler freshness and source/binary hashes. One binary serves both input layouts deliberately; there is no before/after code claim. Preserve raw samples, medians/IQR/worst, planner choices, mapping/topology hashes and exact hardware. PDF and shared archive verification. If relabeling regresses performance, report it and identify next work; do not conceal it by selecting favorable seeds or introducing an unmeasured reorder optimization.

Completed result, 2026-10-20 17:25 +02:00. All 160 cells and 1440 samples passed; all canonical incidence and mapped-rank checks passed. One independently fresh-compiled binary was used across all layouts. Identity inputs won 20/20 paired native comparisons; permuted inputs won 30/60. Chain remained about 1.36x faster, pair about 1.40x faster. Hub changed from 105.58 ms to 283.20–283.88 ms, regions from 71.72 ms to 271.70–272.06 ms; native permuted times were

154.94–155.18 and 146.89–149.59 ms respectively. The range plan gives way to ordered rows after relabeling. Report the limitation and the need for numbering-independent structural planning; no new optimizer added. Local 35 functional checks, Linux 14 client checks/native parity, Clippy and static gates passed. The 519-file remote source snapshot remained unchanged. Own transient services stopped, user's existing toolchain unchanged. User follow-up asks about parallelism and declarative input: this PageRank is serial; parallel query batches already exist. This experiment used generated IR, not Hymeko input files; declarative layout binding is future work through the existing compiler/consumer boundary.

Execution milestones: 2026-10-20 17:40 +02:00

Authority and sequence. User requests fixing vertex-number dependence, then multithreading, then Hymeko-based model/execution handling. Continue the approved kato21 comparison scope after correctness gates; no new profiling, Git publication, checkout or runtime dependency. First ordering milestone about 45–60 minutes plus matched control; parallel execution and declarative source integration follow in that order, with their detailed plans completed before edits. Overall allow several hours. Fresh relevant fetches pass; retain dirty Core/Graph/Tensor checkouts and all prior work.

Ownership and stage one. Core IncidenceProjection owns validated numeric coordinates. Add a borrowed IncidenceLayout view with Canonical or FirstEncounter ordering, sharing the existing row dispatch and ProjectionTargets compression. FirstEncounter assigns active slots in original arc/member visitation order, appending isolates; it is invariant to vertex relabeling with that incidence order fixed. It does not claim invariance to arbitrary hyperedge/member reordering or graph isomorphism. Core identifiers, IR records, coefficients and source data remain unchanged. Tensor RangeWorkspace gains mapped input/output access through its same numeric loops. Graph ExecutionPlan first uses profitable canonical ranges, otherwise evaluates a first-encounter view for higher-arity rows. Its existing component costs and bounded search select ranges; unhelpful layouts are discarded. No new graph or matrix product.

Interfaces and files. Core incidence.rs exports IncidenceLayout/IncidenceOrder; projection.rs delegates row traversal to one shared ProjectionData implementation; projection/layout.rs owns the mapping. Extend prepared_ir_backbone tests. Tensor operator/ranges.rs adds generic mapped coordinate access; virtual_operators tests compare a dense independent oracle. Graph pagerank/plan.rs retains the chosen view and reports its order; pagerank.rs uses it only for speculative range contractions. Existing canonical updates still certify convergence/cap and recover numerical failure. Extend pagerank tests over all three relabel seeds, nonuniform personalization, duplicate slots, empty rows, isolates and iteration caps. Warm runs must retain allocation guarantees.

Cost and risks. First-encounter construction costs O(V+I), with O(V+I) temporary coordinates and compressed retained layout. Pair-heavy cases bypass it. The full-call control includes planning; do not justify regressions by quoting only iteration speed. Preserve canonical output ordering and input coefficient-slot order. Mapped range blocks must not cross omitted components; bounds and numerical fallback are tested. Public methods document coordinate spaces, empty domains and invalid indices.

Stage-one acceptance. Same canonical IR and immutable borrow; exact mapped incidence sequence; label-permuted active layout equality; regained range execution on hub/regions including seeds 17/29/43; dense-rank error bounds and canonical final certification. Run Core projection, Tensor operator and Graph numerical/integration/allocation checks, Clippy/fmt/index/duplication, then Linux smoke and matched current/before/native controls. Native parity remains full-vector/residual; own before/after floating bits may differ because contraction order changes, so retain per-binary determinism and certified errors. Same task-owned local NVMe environment, CPU 2/nice 10, 16 GiB, 180 seconds/process, 15 seconds/operation and other-user/host guards. Immutable snapshots and unique target dirs.

Later stages. Parallel execution must call the same contraction owners, use caller-owned bounded buffers and an explicit worker budget, and retain deterministic reductions/final certification. Avoid global thread-pool changes. Begin with bounded 1/2/4 workers on otherwise idle cores. Declarative handling must compile real .hymeko files through the existing ModuleStore/loader, validate execution policy through its actual consumer, and keep authored identities

separate from internal slots. Inspect the existing language/metamodel/query ownership before choosing the schema; no invented syntax.

Rollback and delivery. Preserve input snapshots before each stage; restore only proven task-owned edits if a candidate fails. Keep all measurements, including losses. Update this four-format plan, write PDF reports and archive with byte verification. No completion claim until all three requested milestones are delivered or an explicit external blocker is documented. Other agents' RL/renderer work remains outside scope.

Ordering milestone result, 2026-10-20. All 120 matched cells/1080 samples passed. Renumbered hub: 284.191 to 117.221 ms; renumbered regions: 270.770 to 106.044 ms. Native: 154.172/147.851 ms. All eight after medians beat native; other cases changed between -0.53 and +0.75 percent. Recovered range execution includes layout construction in measured latency. Remaining renumbering cost is not claimed zero. All source/binary identities retained. The initial remote build was killed when the last SSH session closed (Linger=no), leaving a temporary patched lock. A passive bounded task session now holds the user manager; the exact archive lock was restored and verified. SIGTERM cleanup now restores locks and saves durable input-lock copies; interrupted attempts are retained.

Parallel PageRank: 2026-10-20 18:54 +02:00

Owners and mechanism. Reuse Core's existing reverse-incidence index and canonical transpose fold, which already backs par_arc_to_node. Generalize that fold by OrderedAccumulator and expose a per-node coefficient-policy method through the existing Index/View accessors. No second reverse-index builder or graph representation. Tensor RowGather shares RowTransform's guarded reduction with RowContraction; Core's existing ForwardApplication is replaced by that same gather visitor. Core projection row-range visitation shares the full row dispatcher, preserving uniform-pair specialization and global coefficient offsets.

One solver. Graph keeps one PageRank run/termination/state-update loop. A caller-owned PageRankWorkspace configures a private Rayon pool (default one worker), with no global registry change. Parallel steps gather independent arc masses, then use the existing transpose index to sum each node's contributions in canonical arc/slot order. Outputs are disjoint; no atomic floating sums or per-worker dense node arrays. One reusable arc-value vector is added. The reverse index is lazy, model-owned and checked, never written to canonical IR. Weighted/vector-component, sign/port routing, duplicates, isolates and personalization retain the same policies.

Numerics and resources. Each node/arc reduction remains ordered and compensated. Global mass/change/dangling updates still use the existing canonical vertex sweep. Compare against the canonical scalar step bitwise, and full serial range solves by their certified L1 bounds. Different worker counts on the parallel path must agree bitwise. Iteration caps and numerical errors remain explicit. Test worker count zero, workspace reuse, nonuniform weights, zero destination mass, empty models and shared-model independent workspaces. Bound array allocation by nodes/arcs/incidences and account for the lazy index in retained bytes.

Interfaces and affected files. Add workspace worker configuration and worker-count inspection; PageRankConfig and the existing build/run signatures stay unchanged. Add bounded PageRank errors for pool/index admission. Core incidence.rs, projection.rs and prepared_ir_backbone tests; Tensor operator.rs and virtual_operators tests; Graph pagerank.rs, workspace.rs, private pagerank/parallel.rs, numerical and allocation tests. Existing native evidence client consumes explicit worker settings. Extend the existing ProcessRunner to pin an allowed CPU set before execution, and StructuralControl to record per-subject workers/affinity. No new external dependency.

Measurement. After targeted correctness, Linux smoke, and static gates, compare new 1/2/4-worker execution with the measured serial baseline and one-thread native on chain/pair plus permuted hub/regions. Five process repeats, same raw samples, full rank/residual checks. A multi-worker result is labeled by resource count, never presented as an equal-resource native ranking. Record affinity for the actual Rust workers. Quiet guard covers the selected physical cores and their SMT siblings; nice 10, 16 GiB/180 seconds, at most four compute workers. No compilation overlaps remote timing. Preserve first-milestone snapshots; source handling follows this stage.

Compiled source execution: third milestone, 2026-10-20

Discovery and owner. Transformation fetched successfully on its existing feat/unversioned checkout (fe7d643, two ahead of main), preserving all unrelated dirty work. Formats already owns retained-Core domain/profile projections (environment, metaheuristic, robot_projection) and already depends on Core, Query and Graph. Extend that owner with graph_execution.rs plus a thin graph_rank binary; no new dependency. Core CompiledModel/StdFsProvider/RealParser owns file compilation; QueryEngine owns inherited fields; Graph owns the only solver and execution planning. No new parser, inheritance resolver, IR builder, graph copy, or native benchmark harness.

Input contract. A real .hymeko run profile imports shared defaults and references a graph-source resource declaration carrying a relative file path. The graph file is compiled separately, so profile/default nodes cannot become ranked vertices. Validate exact job path, required typed fields, unknown fields, policy names, positive integral worker counts and Graph's existing numeric controls. Source identity/path handling uses Core canonical paths, not name-prefix inference. Retain both CompiledModels; results keep the graph's canonical NodeId domain.

Declarative ordering. Add Graph RankOrdering and PageRankOptions wrapping the existing PageRankModel (From conversion preserves existing callers). Both build and from_projection accept that one preparation request; no second build entry. Auto retains current cost selection; Canonical excludes first-encounter candidates; FirstEncounter considers that layout only, with the same profitability/fallback gate. Expose the existing config validation method for the source consumer, not a copy. Worker configuration stays caller-owned in PageRankWorkspace.

Deliverables and tests. Curated graph and declaration-permuted .hymeko files, imported profile defaults, serial/parallel run instances, and CLI JSON output with canonical node paths. All authored files go through the Core CLI and real source consumer. Verify equal named graph/ranks across declaration order, actual selected ordering, inherited defaults and overrides, worker selection, unknown/mistyped fields, invalid controls, missing files and clear errors. The source adapter executes only existing Graph APIs; CLI owns argument/output formatting. Allow about 40–60 minutes including integration/CLI checks and a consolidated PDF/archive. No full-source speed claim unless a separate explicit source timing cell is actually executed.

Files and validation. Graph pagerank.rs/plan.rs and exports; Formats lib.rs, graph_execution.rs, bin/graph_rank.rs, data/graph_execution files and focused integration tests. Use temporary CLI patches for current Core/Graph/Tensor, restore lockfiles, and audit duplicate crate identities. Existing query/geometry/emitter changes belong to the user and must not be overwritten. Any unrelated pre-existing format/check failure is recorded separately. Tensor's whole-repo fmt currently flags two untouched bounded-planning files; verify their baseline hashes and use scoped format checks for this task, without silently rewriting them.

All three milestones delivered, 2026-10-20 20:29 +02:00. Ordering: 120 processes and 1080 samples; permuted hub/regions improved 58.75/60.84 percent, eight after medians below native. Parallel: 100 processes/900 samples; new serial/two/four worker times: chain 19.877/26.651/21.425 ms, pair 123.906/120.451/85.945, hub 116.330/179.736/113.204, regions 104.778/192.077/123.642. Default remains one worker; more workers are explicit. All sampled thread affinities verified. Source: five real Hymeko files compile via Core CLI, five profile jobs execute, two integration and one schema test pass. Named ranks and canonical graph hashes agree across authored declaration order. The source endpoint is functionally checked on Windows, not independently timed as a large-file Linux workload. Final Graph regressions and package Clippy pass; Source consumer/CLI rebuilt after the common finish refactor. Graph duplication is 547 groups/9129 lines

against unchanged 547/12629 baseline. Core checks pass; Tensor has two verified pre-existing untouched global-fmt differences; Geometry has four pre-existing dependency warnings. No dependencies, commits, pushes or extra worktrees added. Local integration used restored CLI patches; publishing provider revisions/consumer pins is separate. Consolidated six-page PDF, snapshots, binaries, raw data and scoped diffs are retained and archived with content verification. Owned remote compute services stopped; passive session stop marker issued.

Sparse-row deduplication follow-up, 2026-10-20 20:43 +02:00

The user identified repeated SparseRow implementations. Move the existing private Core IndexedRow, unchanged numeric kernels and singleton range behavior, into Tensor's existing operator/accumulator module and export a borrowed constructor. Remove the test-only Row numerical implementation; literal fixtures use IndexedRow. Keep Core RunRow for compressed traversal and IncidenceRow for representation dispatch. Core projection calls the shared Tensor implementation. Independent literal and dense oracles stay independent; test long-interval contraction on the actual Core RunRow. No new dependencies, IR or allocation; pair specialization, slot offsets, duplicate order, lazy failure/partial writes and override dispatch must remain unchanged. Files: Tensor operator.rs, operator/accumulator.rs, tests/virtual_operators.rs; Core incidence/projection.rs, tests/prepared_ir_backbone.rs. Focused Tensor, Core, Graph PageRank correctness and allocation contracts, scoped formatting and Clippy. No performance measurement requested. Estimated 15–25 minutes including checks. Rollback only this follow-up delta from the local pre-edit snapshot. Discovery: four impls in these owners, existing IndexedRow; symbol index has no public IndexedRow. Preflight's Index/index candidates are resolver/index APIs, not sparse coordinate rows. Extend the existing Tensor owner and remove the Core/test copies.

Follow-up verified, 2026-10-20 20:47 +02:00. Shared IndexedRow implementation matches the pre-move body after trait-name normalization; the test-only implementation is removed. IncidenceRow range visitation also delegates to the storage implementations. Tensor 8, Core 23 and Graph 20+6 targeted correctness/allocation tests pass; Core's timing test stays ignored. All three package Clippy checks and touched-file formatting pass. Core duplication gate: 458 groups/8730 lines, unchanged baseline 484/10411. Tensor inventory: 155 groups/2436 lines (no configured baseline). No benchmark, new dependency, publication or remote job. Local CLI dependency patches were restored after testing; existing manifest revision pins remain unchanged.

Graph algorithms and generators, 2026-10-20 22:11 +02:00

Scope: audit existing algorithms and generation, run focused correctness suites, and exercise Core combinatorial designs through the existing Case/IrConstruction adapter on the real graph algorithms. Extend steiner_mycielski_nav.rs with a private runner: independent reachability and shortest-path oracles, connected components, serial/parallel batch parity, dual reachability and Levi rank/basis identities. Original and seeded vertex relabelings share the existing VertexPermutation. Retain existing weighted/directed, s-connectivity and bounded-enumeration suites. Replace informed_common's copied Mycielski recurrence by Core mycielskian, preserving ordered endpoints. Capture old ordered membership fingerprints before replacement. Owners: Core generates combinatorial topology; Graph executes algorithms; test-only adapters feed IrConstruction. No new production model, algorithm or dependency. Verification: existing 84 checks passed; run the new matrix, all affected shared-helper clients as justified, package Clippy/scoped formatting, and unchanged duplication gate. Timing/profiling remains off pending explicit measurement selection. Small bounded correctness graphs and the existing STS(1023) formula checks; no remote jobs. Estimated 15–25 minutes including validation and a concise PDF/archive. Rollback

only this phase's diff using the recorded pre-edit snapshots. Discovery: existing generator and oracle owners reused. Mycielski copy is removed; Steiner XOR projective construction differs from Core Bose/backtracking and is not silently substituted. Older unowned-arc fixtures remain clearly distinguished from full model generation. Preserve exact query semantics and incidence multiplicity.

Kato21 algorithm measurements, 2026-10-20 23:44 +02:00

Authorization: user explicitly selected measurements on kato21 for the algorithm and generator investigation. Extend existing Fixture/Core generation adapters, ranking evidence client and Campaign; no new production algorithm or graph builder. Add Core Bose STS(255), complete 3-uniform (32), sunflower (1024 petals, core 1, 32 exclusive members) and seven-step Mycielskian of C5 to the existing catalog. Compare with chain65k, hub32_1m and components8_1m. Separate canonical generation (excluding destruction), reverse index construction, warm reachability, dual reachability, shortest Berge/BFS, Dijkstra and Levi basis. Matched Hypergraphz comparison covers connected components, with full partition parity. Measure s=2 overlap components only on bounded combinatorial workloads: preflight sum of distinct incidence-degree pairs must not exceed five million. Five fresh processes per selected cell, nine raw samples, two warmups; reusable ProcessRunner, explicit opt-in, exact source and binary digests, caller-independent scratch boundaries. Keep old default matrices; explicit selection opts into additions. Source snapshots must work without a Git checkout; manifest bytes are authoritative. QuietHostGuard before each cell, CPU 2/SMT sibling 66, nice 10, idle I/O, 16 GiB/180-second process and 15-second operation budgets. Do not disturb other users. Run representative full-size pilot cells before the matrix. Initial expected measurement time 25–40 minutes from the existing 250 ms sample calibration and one-second quiet gates, plus fresh compilation; reconcile with pilot observations. Keep a bounded passive SSH session while user services run (Linger=no); stop only owned services and close it on completion. Reuse existing private NVMe scratch, Rust 1.97.1, vendored dependencies and pinned native library. No install/upgrade. Correctness gates: generator dimensions/ordered memberships, Core index validation, small query/basis/s-overlap known answers; Python selection, metadata and resume contracts; previous Graph checks and current Linux parity before timing. Produce PDF, numerical tables, plots, raw logs, manifests and verified Samba archive. Rollback only task-owned deltas from initial snapshots; retain failed pilot evidence.

Kato21 algorithm phase completed, 2026-10-21 00:51 +02:00. Fourteen separate pilot processes passed. Main measurements: 330 processes, 2970 raw samples, 66 workload/operation combinations, five repeats each. Connected components: HyMeKo below Hypergraphz in all seven medians and all 35 paired repeats, ratios 1.879–14.936. Full raw-log/summary and partition parity checks passed; all sampled thread affinities verified. Source snapshot: 519 files; binary 0fec5d07e4253920cbe8f6132562bcd3c0bde1f6388bbb755d97e0d8f109ab8d. Highest process RSS 204.98 MiB; longest process 7.66 seconds. S=2 overlap timings 130.66/227.98/20.80 ms identify the next algorithmic target; alternative candidate counts are estimates, not measured speedups. Host account UID 5069 triggered strict quiet gates; only SSH/session services were observed, no other human work inferred. All owned compute services and the passive SSH session stopped; original user Rust unchanged. Current local source bytes match the measured snapshot. Source/API changes are confined to measurement/test adapters and existing controllers; no dependency or production graph algorithm changed in this phase. Four-page PDF and raw artifacts: Graph reports/2026-10-19-structural-pagerank/algorithm-generation/nitech-kato21.